

CÁLCULO DEL ANCHO DE BANDA

Ancho de banda en VoIP

Los DSP convierten señales de voz analógica a señales de voz digital a través de un códec determinado. El DSP genera una serie de bits por segundos, una muestra se obtiene cada 10 milisegundos cuyo tamaño dependerá del tipo de códec utilizado. La siguiente tabla muestra cómo puede variar el tamaño de la muestra según el códec que se utilice.

Ancho de banda del códec	Tamaño de la muestra digital en bits (10ms)	Tamaño de la voz digitalizada (10 ms)	Tamaño de dos muestras digitalizadas (20 ms)
G.711: 64 Kbps	$64,000 \text{ bps} \times 10/1000$ sec = 640 bits	80 bytes	$2 \times 80 = 160$ bytes
G.726 r32: 32 Kbps	$32,000 \text{ bps} \times 10/1000$ sec = 320 bits	40 bytes	$2 \times 40 = 80$ bytes
G.726 r24: 24 Kbps	$24,000 \text{ bps} \times 10/1000$ sec = 240 bits	30 bytes	$2 \times 30 = 60$ bytes
G.726 r16: 16 Kbps	$16,000 \text{ bps} \times 10/1000$ sec = 160 bits	20 bytes	$2 \times 20 = 40$ bytes
G.728: 16 Kbps	$16,000 \text{ bps} \times 10/1000$ sec = 160 bits	20 bytes	$2 \times 20 = 40$ bytes
G.729: 8 Kbps	$8000 \text{ bps} \times 10/1000 \text{ sec}$ = 80 bits	10 bytes	$2 \times 10 = 20$ bytes

El tamaño total de la trama de capa 2 encapsulando un paquete de VoIP depende de los siguientes factores:

- Velocidad y tamaño del paquete, se mide en pps (paquetes por segundo) es inversamente proporcional al tamaño del paquete (expresado en bytes).
- Sobrecarga de la cabecera IP, se refiere al número total de bytes en las cabeceras RTP, UDP e IP. Sin compresión la sobrecarga es de 40 bytes.

- Sobrecarga de la cabecera de la capa de enlace, el tamaño depende del tipo de encapsulación o el tipo de trama. Para el caso de Ethernet es de 18 bytes, y para 802.1q, 22 byte.
- Sobrecarga de los túneles, únicamente para los casos donde se utilizan, **GRE** (Generic routing encapsulation), **L2TP** (Layer 2 Tunneling Protocol), **IPsec** (IP security), **802.1Q** y **MPLS** son técnicas de tunelización comunes cada una con una sobrecarga en particular.

Los códecs pueden ser de diferentes tipos, el tamaño de cada paquete de VoIP depende del tipo de códec utilizado y el número de muestra encapsulada dentro de cada paquete de VoIP. La siguiente lista describe alguno de ellos:

- **G.711 PCM**
- **G.726 ADPCM** (adaptive differential pulse code modulation)
- **G.722** (wideband speech encoding standard)
- **G.728 LDCELP** (low delay code exited linear prediction)
- **G.729 CS-ACELP** (conjugate structure algebraic code exited linear prediction)

Los DSP producen una muestra digital de 10 milisegundos de señal de voz analógica, esto es común entre dispositivos de voz Cisco, también es posible poner más de dos muestras de VoIP si es necesario. El período de enpaquetamiento es la suma de la señal analógica de voz expresada en milisegundos que es encapsulada en cada paquete IP en formato digitalizado.

El problema añadido de sumar muchas muestras de voz digital en un paquete IP es que cuando el paquete es descartado se pierde demasiada voz en el proceso. Dicha pérdida tiene un efecto negativo en la calidad de la llamada mayor que cuando se pierden simples paquetes. Además de que se añade un retraso extra.

Sobrecarga de la capa de enlace

El transmitir un paquete IP sobre un enlace requiere encapsular el paquete IP en una trama que sea apropiada para dicho enlace y a través del protocolo adecuado para ese enlace. Cualquier paquete que sea transmitido fuera de una determinada interfaz debe ser encapsulado según el protocolo que ésta esté utilizando.

Cuando un router recibe un paquete puede entrar con un cierto tipo de encapsulación como por ejemplo Ethernet, el paquete es desencapsulado y se inicia el proceso de enrutamiento. Una vez determinada la interfaz de salida se encapsula en la trama para la interfaz adecuada y su respectiva configuración. Finalmente el paquete puede dejar el router a través de una interfaz distinta como puede ser PPP.

La sobrecarga de la capa de enlace para Ethernet, Frame-Relay, MLP y dot1Q es 18, 6, 6 y 22 respectivamente. Durante el cálculo del ancho de banda total requerido para una llamada de VoIP por cada tipo de enlace se debe considerar la sobrecarga de la capa de enlace adecuada.

Sobrecarga debida a seguridad y tunelización

IPsec (IP security) es un conjunto de protocolos estándar de la IETF que permiten la transmisión segura de información. IPsec puede operar en dos modos, modo transporte y modo túnel. En modo transporte la encriptación es aplicada solamente a la carga útil del paquete IP mientras que en modo túnel la encriptación se aplica a todo el paquete, incluyendo la cabecera. Cuando la cabecera es encriptada los routers intermedios no pueden leer su información, de manera que es necesario el uso de una segunda cabecera para que estos routers puedan leer cierta información. Esta nueva cabecera supone 20 bytes extra de sobrecarga. Además tanto en modo transporte como modo túnel es posible utilizar **AH** (Authentication Header) para proporcionar autenticación y/o **ESP** (Encapsulating Security Payload) para proporcionar autenticación y encriptación. Todo esto ha de tenerse en cuenta a la hora de calcular los requerimientos de ancho de banda necesarios para VoIP.

Existen otros protocolos que tunelizan el tráfico sin añadir seguridad y que son los siguientes:

- **GRE** (Generic Routing Encapsulation). Transporta datos de capa 2 o de capa 3 sobre redes IP.
- **L2F/L2TP** (Layer 2 Forwarding / Layer 2 Tunneling Protocol). Transportan tramas PPP sobre IP.
- **PPPoE** (PPP over Ethernet). Transporta tramas PPP sobre Ethernet.
- **QinQ**. Transporta tramas dot1q encapsuladas en una segunda VLAN ocultando a los switches la primera VLAN ID.

La utilización de cualquiera de estos protocolos de tunelización se debe tener muy en cuenta a la hora de calcular los requerimientos de ancho de banda para VoIP.

A continuación se muestran los tamaños de sobrecarga de IPsec y de los principales protocolos de tunelización.

Protocolo	Tamaño de la cabecera
IPsec Transport Mode Con DES o 3DES y MD5 y SHA-1 para encriptación y autenticación.	30 a 37 bytes
IPsec Transport Mode Con ESP y AES para encriptación y AES-XCBC para autenticación.	38 a 53 bytes
IPsec Tunnel Mode	50 a 57 bytes o 58 a 73 bytes
L2TP	24 bytes
GRE	24 bytes
MPLS	4 bytes
PPPoE	8 bytes

Cálculo del ancho de banda total para una llamada de VoIP

Existen una serie de propósitos para los cuales es necesario conocer el ancho de banda que ocupan las llamadas de VoIP, como por ejemplo los siguientes:

- Diseño y planificación de la capacidad de los enlaces.
- CAC
- QoS

Para calcular el ancho de banda consumido en una llamada de VoIP es necesario determinar las siguientes cuestiones:

- Determinar el códec y el período de encapsulación. Diferentes códecs generan distintos números de bits por segundo, generalmente en un rango entre 5,3 Kbps y 64 Kbps. El número de muestras de voz digital encapsuladas en cada paquete IP determina el período de encapsulación. Para dispositivos de Voz Cisco el período que lleva empaquetar es de 20 ms, por lo que cada paquete de VoIP encapsulará dos muestras de 10 ms de voz digital.
- Determinar información específica al enlace. Verificar si cRTP es utilizado y si hay tunelización o seguridad aplicada.
- Calcular el tamaño de empaquetamiento. Para ello se multiplica el ancho de banda del códec por el período que lleva empaquetar y se divide el resultado entre 8. Así se obtiene el tamaño de la carga útil de voz en bytes.
- Calcular el tamaño total de la trama. Se suman el tamaño de las cabeceras IP, UDP, RTP o cRTP, más las cabeceras opcionales de tunelización y de la capa de enlace vista en el paso segundo. Todo se agrega a la carga útil de voz calculada en el paso tercero. El resultado es el tamaño total de la trama.
- Calcular el volumen de paquetes. El volumen de paquetes es inverso al período de empaquetamiento (convertido a segundos). En otras palabras, dividir 1 entre el período de empaquetamiento.
- Cálculo del ancho de banda total. Se calcula multiplicando el tamaño total de la trama (paso cuarto) convertido a bits por el volumen de paquetes (paso quinto).

La siguiente figura muestra una trama de VoIP y dos métodos para calcular el ancho de banda requerido para una llamada. El primer método está basado en los 6 pasos descritos anteriormente. El segundo método se aplica en el tamaño total de la trama sobre la carga útil, que es igual al ancho de banda requerido sobre ancho de banda de los códecs.

E	D	C	B	A
Cabecera capa 2	Cabecera túnel	Cabecera seguridad	Cabeceras IP+UDP+RTP o cRTP	Voz digitalizada (El tamaño depende del tipo de códec y de la encapsulación del paquete IP)

Método 1:

$$A \text{ (cantidad de voz digitalizada en bytes)} = \frac{\text{Códec (bps)} \times \text{Período de empaquetamiento (seg)}}{8 \text{ bytes}}$$

$$F \text{ (tamaño total de la trama en bits)} = 8 \times (E + D + C + B + A)$$

$$R \text{ (volumen del paquete)} = \frac{1}{\text{Período de empaquetamiento (seg)}}$$

$$\text{Ancho de banda (kbps)} = \frac{F \times R}{1000}$$

Método 2:

$$A \text{ (cantidad de voz digitalizada en bytes)} = \frac{\text{Códec (bps)} \times \text{Período de empaquetamiento (seg)}}{8 \text{ bytes}}$$

$$F \text{ (tamaño total de la trama en bytes)} = (E + D + C + B + A)$$

$$\text{Ancho de banda} = \frac{\text{Códec (bps)} \times F}{A}$$

$$\frac{\text{Tamaño total}}{\text{Tamaño carga útil}} = \frac{\text{Ancho de banda requerido}}{\text{Ancho de banda del códec}}$$

Una vez calculado el ancho de banda necesario para una llamada de VoIP se puede calcular el ancho de banda total que se reservará para VoIP teniendo en cuenta el máximo número de llamadas concurrentes que se necesitan tener.

Detección de la actividad de voz

VAD (Voice Activity Detection) es una función habilitada en redes de VoIP que sirve para detectar silencios (pausas en el diálogo) y audio en una sola dirección de tal manera que no genera datos, ahorrando así ancho de banda. En una conversación normal hasta una tercera parte de la misma son silencios.

Cuando alguien pone música en el teléfono (MOH) en un período de espera, mientras se espera solamente se envía voz en una dirección.

La cantidad de ancho de banda ahorrado con VAD depende de los siguientes factores:

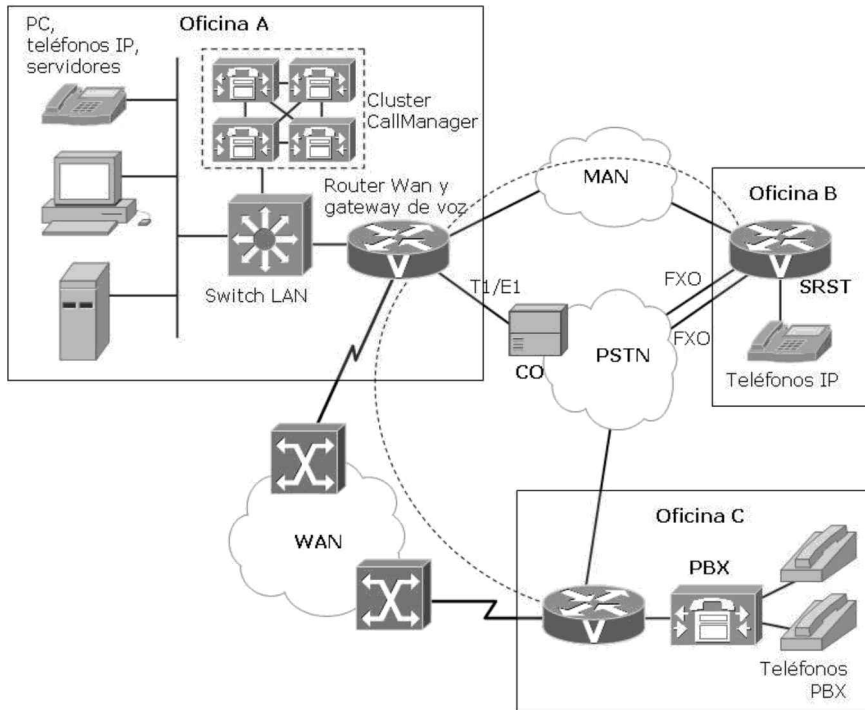
- Durante una conversación telefónica normalmente sólo una persona habla a la vez, de tal manera que la persona que escucha no necesita enviar datos. Lo mismo se aplica a quien está recibiendo MOH.
- Si el ruido de fondo es alto VAD no puede detectar silencio y no se produce ahorro de ancho de banda.
- Existen factores varios durante la llamada que hacen que VAD pueda gestionar mejor o peor los recursos. Por ejemplo durante una sesión *e-learning* en la que solamente una persona envía voz VAD puede ahorrar una cantidad considerable de ancho de banda.

Los estudios dicen que la media de ancho de banda que se puede ahorrar con el uso de VAD es de alrededor de un 35%.

IMPLEMENTACIÓN DE VoIP EN UNA RED EMPRESARIAL

Los principales elementos relativos a telefonía en una implementación empresarial de VoIP de Cisco son: gateway, gatekeeper, Cisco Unified CallManager y teléfonos IP Cisco. Los teléfonos necesitan al CallManager, que actúa como una PBX. Los gateways proporcionan conectividad entre analógico, digital, dispositivos de telefonía IP y circuitos. El gatekeeper es un dispositivo H323 que proporciona enrutamiento de llamadas o servicios CAC.

Las implementaciones de VoIP pueden variar dependiendo de muchos factores. La siguiente figura muestra una empresa con tres oficinas remotas:



En la oficina A funciona una implementación de servicios de telefonía IP. Dicha oficina cuenta con un clúster de Cisco Unified CallManager y todos los empleados usan teléfonos IP. Está conectada a la oficina B mediante una red MAN de tipo Metro Ethernet, y a la oficina C usando un enlace WAN tipo ATM o Frame-Relay. En caso de que la MAN o WAN fallen, las llamadas deberán ser enrutadas a través de la PSTN, al igual que en caso de congestión (haciendo uso de Automated Alternate Routing, **AAR**). Las llamadas hacia o desde el exterior de la empresa son realizadas mediante la PSTN.

La oficina B está conectada a la oficina A mediante una red MAN de alta velocidad y sus teléfonos IP están gestionados por el clúster de Cisco Unified CallManager en la oficina A. Las llamadas de VoIP entre las oficinas B y A se efectúan a través de la red MAN. Las llamadas entre las oficinas B y C van por la red MAN hasta A y después por la WAN hasta C. Las llamadas de la oficina C a B usarán el camino inverso. Si el enlace MAN se cae se utilizaría Survivable Remote Site Telephony (SRST), que está implementado en el Gateway y permite que los teléfonos de la oficina B puedan comunicarse entre sí, además permite una llamada externa a través de la PSTN. La figura muestra que el gateway tiene dos interfaces FXO, una está conectada al número de emergencias dejando el otro puerto para una llamada externa.

La oficina C cuenta con una vieja PBX y teléfonos clásicos. Un gateway de voz proporciona conectividad entre la propia oficina C y la PSTN y también con el resto de los teléfonos en otras oficinas a través de la WAN. El tráfico de voz normalmente usará el camino a través de la WAN a no ser que ésta esté caída, entonces usará la PSTN. Todas las llamadas externas a la empresa desde y hacia C serán cursadas por la PSTN.

Gateway de voz en un router Cisco

Los gateway proporcionan conectividad entre interfaces analógicas como por ejemplo FXO, FXS, interfaces digitales como T1/E1, BRI y dispositivos de telefonía IP como por ejemplo fax, teléfono analógico.

Los gateway convierten señales digitales a analógicas y viceversa. También pueden ser capaces de manejar distintos tipos de códecs. Estas capacidades dependen de los DSP instalados en el gateway y en las características de la IOS. **SRST** (Survivable Remote Site Telephony) es una función de IOS realmente útil en sitios donde los teléfonos IP están gestionados desde otra oficina, por ejemplo desde la oficina central. En caso de pérdida de conectividad con la oficina central los teléfonos quedarían inutilizados, a no ser que el gateway tuviera la función SRST, entonces esos teléfonos podrían comunicarse entre sí, mantener ciertas funciones y enrutar un número de llamadas limitadas por la PSTN.

En caso de que la IOS permita implementar Cisco CallManager Express es posible tener un dispositivo que podría ser el gateway actuando como CallManager, esta función es útil en sitios remotos pequeños.

Además de las características vistas los gateway pueden ofrecer más servicios dependiendo del software que ejecuten.

Cisco Callmanager

CCM (Cisco CallManager) es un software de procesamiento de llamadas y el principal componente de Cisco Unified Communication System. CCM soporta los protocolos de señalización MGCP, H323, SIP y SCCP.

Las siguientes son varias de las funciones que Cisco Unified CallManager proporciona:

- Procesamiento de llamadas. CCM lleva a cabo enrutamiento de llamadas, señalización y contabilización. Además tiene capacidades de gestión de ancho de banda y CoS.

- Administración del plan de llamadas. CCM actúa como el CA para gateways MGCP y teléfonos IP. El plan de llamadas se administra e implementa en el CCM, liberando a sus clientes de necesitar esa información.
- Control y señalización de dispositivos. CCM realiza la señalización para sus clientes y controla su configuración y comportamiento.
- Administración de las funcionalidades de los teléfonos. Los archivos de configuración de los teléfonos IP se guardan en el CCM, de manera que el cliente al inicializarse obtiene la configuración del CCM.
- Servicios de directorio y XML. Estos servicios pueden estar disponibles desde el CCM. Los teléfonos pueden hacer búsquedas en el directorio y los dispositivos XML pueden ser gestionados como si se tratara de teléfonos IP.
- Interfaz de programación para aplicaciones externas. Cisco Systems proporciona una **API** (Application Programming Interface) para ser usada por aplicaciones externas y que así puedan trabajar y comunicarse con Cisco Unified CallManager.

Modelos de despliegue VoIP

Existen varios modelos de despliegue usando Cisco Unified CallManager:

- **Sitio simple.** Tal y como el nombre indica la empresa tiene solamente un sitio donde se implementa un clúster de Cisco CallManager. Los teléfonos IP y los posibles gateways MGCP están gestionados por el CCM, quien puede comunicarse con dispositivos H323 y SIP. Las llamadas desde y hacia el exterior son cursadas a través de la PSTN.
- **Múltiples sitios con procesamiento de llamadas centralizado.** En este escenario existen múltiples sitios y en uno de ellos, normalmente el principal o central, CCM está implementado. Los teléfonos IP y posibles gateway están gestionados por el CCM que está localizado en el sitio central. Aquí el tráfico de señalización siempre viajará a través de los enlaces que conectan los sitios, incluso para llamadas locales dentro de un sitio. El tráfico de VoIP no viajará por dichos enlaces si la llamada es local. Normalmente cada sitio tiene una conexión PSTN para realizar y recibir llamadas

externas o para ser usado en caso de que el enlace con la oficina principal se caiga.

- **Múltiples sitios con procesamiento de llamadas distribuido.** En este caso cada sitio tiene su propio clúster de Cisco CallManager. Cuando se cursan llamadas dentro de cada sitio el tráfico de señalización y VoIP no viaja por los enlaces WAN.
- **Clúster sobre enlaces WAN.** En este modelo el clúster de Cisco CallManager está expandido en diferentes sitios y se comunica vía enlaces WAN. Para que funcione adecuadamente el máximo retraso RTT no ha de ser superior a 40 ms.

CALIDAD DE SERVICIO

CONVERGENCIA DE RED Y QoS

Actualmente las redes soportan diferentes tipos de aplicaciones tales como voz, vídeo y datos sobre una infraestructura común. La convergencia de todos estos tipos de aplicaciones conjuntas representa un reto para el personal administrador encargado de ello.

El retardo aceptable para un paquete de **VoIP** es de 150 a 200 ms y el **jitter** o variación en el retardo ha de ser mínimo para mantener la consistencia de la llamada. Por el contrario, una aplicación de datos tal como FTP no es tan sensitiva al retardo y el jitter ni siquiera sería apreciado. Cuando conjuntamente existen voz y datos en la red se deben usar mecanismos para hacer que puedan coexistir.

Muchas aplicaciones de datos están basadas en protocolos orientados a la conexión como TCP, cuando pierden un segmento otro es retransmitido, mientras que las aplicaciones de voz o vídeo tienen una tolerancia mínima hacia las pérdidas de datos. Debido a esto es necesario implementar mecanismos que prioricen determinados tipos de tráfico cuando exista congestión en la red.

Los fallos en la red afectan a todas las aplicaciones, mientras la red converge después de un fallo quienes más sufren el desperfecto son los usuarios que estén usando aplicaciones interactivas de voz o vídeo, pudiendo incluso perder la llamada.

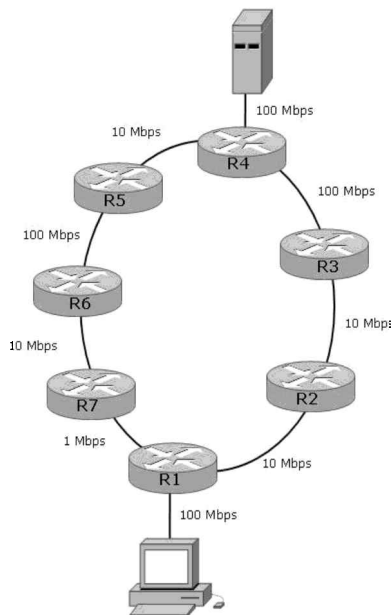
Existen cuatro cuestiones importantes a tener en cuenta en redes convergentes:

- Ancho de banda disponible.
- Retraso de extremo a extremo.
- Jitter o fluctuación en el retraso.
- Pérdida de paquetes.

Ancho de banda disponible

Los paquetes normalmente fluyen usando el camino con mejor ancho de banda. El mejor ancho de banda disponible en la ruta es el del enlace con menor ancho de banda. En la siguiente figura se observa que la ruta a través de los router R1-R2-R3-R4 es el mejor camino del cliente al servidor, y 10 Mbps es el máximo ancho de banda del camino.

El ancho de banda disponible es el máximo ancho de banda de la ruta dividido entre el número de flujos.



La falta de ancho de banda hace que las aplicaciones se degraden debido al retraso y a la pérdida de paquetes. Esto es detectado de forma inmediata por los usuarios de aplicaciones de voz o vídeo.

Es posible resolver los problemas de ancho de banda con algunos de los siguientes recursos:

- Incrementar el ancho de banda. Lo cual es efectivo pero costoso. Aunque dependiendo del escenario en algunos casos es recomendable.
- Usar mecanismos de **QoS** de clasificación y marcado así como mecanismos de encolamiento apropiados. De esta manera se enviarán primero los paquetes más importantes.
- Usar técnicas de compresión. Compresión a capa 2, compresión de cabeceras TCP, **cRTP** (RTP header compression), etc. Siempre es preferible compresión en hardware en vez de en software, ya que el mecanismo en sí usa muchos recursos de CPU.

Retraso de extremo a extremo

Hay diferentes tipos de retraso desde origen a destino. El retraso de extremo a extremo es la suma de estos cuatro tipos de retraso:

- **Retraso de procesamiento**, es el tiempo que un dispositivo de capa 3 tarda en mover un paquete desde la interfaz de entrada a la de salida. El tipo de CPU así como la arquitectura de hardware influyen en esto.
- **Retraso de encolamiento**, es el tiempo que un paquete pasa en la cola de salida de una interfaz. Dependerá de lo ocupado que esté el router, del número de paquetes esperando, del tipo de cola y del ancho de banda de la interfaz.
- **Retraso de serialización**, es el tiempo empleado en poner en el medio físico todos los bits de una trama.
- **Retraso de propagación**, es el tiempo que se tarda en transmitir en el medio físico los bits correspondientes a una trama. Depende del tipo de medio físico.

Variación del retraso

Las fluctuaciones en el retraso reciben el nombre de **jitter**. Se produce cuando los paquetes llegan al destino a velocidades diferentes a las que se emitieron desde el origen. Para paquetes de **VoIP** o vídeo es esencial que la aplicación sea capaz de liberarlos en el destino a la misma velocidad y en el mismo orden que fueron emitidos en un principio. Esto lo hace sirviéndose del buffer, que es donde se van almacenando a medida que llegan y de **RTP** (Real-Time Transport Protocol) que sella los paquetes para que sean entregados en orden.

Algunas de las claves para ayudar a reducir el jitter son las siguientes:

- Incrementar en ancho de banda.
- Priorizar los paquetes sensitivos.
- Usar técnicas de compresión de capa 2.
- Usar técnicas de compresión de cabeceras.

Pérdida de paquetes

La pérdida de paquetes ocurre cuando un router no tiene más espacio libre en el buffer de memoria de la interfaz de salida para almacenar los nuevos paquetes que le llegan, debiendo descartarlos.

TCP reenvía los paquetes descartados, a la vez que reduce el tamaño de ventana. Aplicaciones UDP como TFTP por ejemplo pueden generar más tráfico en la red al tener que retransmitir un archivo completo en caso de pérdida de paquetes. Para llamadas de VoIP la pérdida de paquetes resulta en conversaciones entrecortadas, mientras que para vídeo la imagen parece congelarse. Con los mecanismos de QoS adecuados es posible evitar estas situaciones.

- A través del comando **show interface** es posible obtener información cuando existan pérdidas de paquetes o congestión:
- **Output drop**: número de paquetes descartados, debido a que la cola de salida de la interfaz está llena.
- **Input queue drop**: si la CPU está sobrecargada el router podría tener problemas al procesar paquetes entrantes, incrementando este contador.

- **Ignore:** número de tramas ignoradas debido a falta de espacio en el buffer.
- **Overrun:** cuando la CPU está sobrecargada podría no proporcionar espacio en el buffer lo suficientemente rápido, haciendo que se descarten paquetes.
- **Frame error:** incluye las tramas con CRC no válido, las que son más pequeñas que el standard (runts) y las gigantes (giants).

Los siguientes métodos pueden utilizarse para reducir o evitar la pérdida de paquetes:

- Incrementar el ancho de banda.
- Incrementar el tamaño del buffer. Modificando los valores por defecto.
- Proporcionar un ancho de banda garantizado. Usando herramientas de QoS tales como **CBWFQ** (Class Based Weighted Fair Queuing) o **LLQ** (Low Latency Queuing).
- Evitar la congestión. Descartando aleatoriamente paquetes antes de que las colas se llenen. Para esto existen métodos como **RED** (Random Early Detection) y **WRED** (Weighted Random Early Detection).

IMPLEMENTACIÓN DE QoS

QoS se define como la habilidad de la red para proporcionar un mejor o especial servicio a un conjunto de usuarios o aplicaciones en detrimento de otros usuarios o aplicaciones.

Para implementar QoS hay que llevar a cabo tres pasos:

- Identificar tipos de tráfico y sus requerimientos.
- Clasificación del tráfico basándose en los requerimientos identificados.
- Definir las políticas para cada clase.

Identificación del tráfico y sus requerimientos

Es el punto de partida en cualquier implementación de QoS y conlleva los siguientes apartados:

- Llevar a cabo una auditoría de red. Es aconsejable tomar estos datos durante los momentos en que la red esté más ocupada así como durante otros períodos.
- Determinar la importancia de cada aplicación. El modelo de negocio determinará la importancia de cada aplicación. Se pueden definir clases de tráfico y los requerimientos para cada clase.
- Definir niveles de servicio para cada clase de tráfico. Cada clase identificada previamente ha de tener un nivel de servicio que constará de características como ancho de banda garantizado, retraso, preferencia a la hora de que se descarte, etc.

Clasificación del tráfico

Es posible clasificar desde unas pocas a cientos de variaciones de tráfico dentro de diferentes clases. Las clases definidas han de ir de acuerdo a las necesidades y objetivos de negocio. Las siguientes clases son el resultado de varios estudios y está demostrado que sino todas, alguna de ellas aparecerá en cualquier red empresarial:

- **Clase de VoIP**, como su propio nombre indica corresponde al tráfico de VoIP.
- **Clase de aplicaciones de misión crítica**, corresponde a aplicaciones de alta importancia.
- **Clase de tráfico de señalización**, pertenece al tráfico de señalización de VoIP, vídeo, etc.
- **Clase de tráfico de aplicaciones de transacción**, son aplicaciones del tipo de bases de datos interactivas, etc.
- **Clase Best-effort**, esta clase engloba el tráfico no estipulado en las anteriores y se le proporciona el ancho de banda que sobre.
- **Clase sin importancia**, corresponde a servicios o aplicaciones que se consideran inferiores a las Best-effort. Podrían ser e-mail personal, aplicaciones P2P, juegos online, etc.

Definición de políticas para cada clase

Este paso conlleva el completar las siguientes tareas:

- Especificar un ancho de banda máximo.
- Especificar un ancho de banda mínimo garantizado.
- Asignar niveles de prioridad.
- Usar herramientas que sean adecuadas para la congestión gestionándola, eliminándola, etc.

La tabla siguiente muestra un ejemplo de una política de QoS.

Clase	Prioridad	Tipo de cola	Ancho de banda Mín/Máx	Herramienta
Voice	5	Prioridad	1 Mbps Mín 1 Mbps Máx	Prioridad de cola
Business mission critical	4	CBWFQ	1 Mbps Mín	CBWFQ
Signaling	3	CBWFQ	400 Kbps Mín	CBWFQ
Transactional	2	CBWFQ	1 Mbps Mín	CBWFQ
Best-effort	1	CBWFQ	500 Kbps Máx	CBWFQ CB-Policing
Scavenger	0	CBWFQ	Máx 100 Kbps	CBWFQ CB-Policing WRED

MODELOS DE QoS

Modelo Best-Effort

El modelo Best-Effort significa que no hay QoS aplicado, de manera que todos los paquetes dentro de la red independientemente del tipo que sean reciben el mismo trato. Como beneficio de este sistema está la facilidad de implementación, ya que no hay que hacer nada para ponerlo en funcionamiento, pero tiene como

desventaja que no es posible garantizar ningún tipo de servicio a ninguna aplicación.

Modelo de servicios integrados

Se trata del primer modelo que proporcionó QoS de extremo a extremo y basado en la señalización explícita y reserva de recursos de red para aquellas aplicaciones que los necesitan. El protocolo usado para la señalización es el **RSVP** (Resource Reservation Protocol). Cuando una aplicación tiene un requerimiento de ancho de banda RSVP va salto por salto a lo largo del camino intentando hacer la reserva solicitada en cada uno de los routers que se encuentra en la ruta. Si la reserva se puede hacer la aplicación podrá operar; pero si algún elemento en el camino no tiene los recursos suficientes la aplicación tendrá que esperar.

Para implementar Servicios Integrados de manera satisfactoria además de RSVP debería habilitarse lo siguiente:

- **Control de Admisión**, en caso de que los recursos no puedan proporcionarse sin afectar a las aplicaciones actualmente en uso se deberían denegar.
- **Clasificación**, el tráfico perteneciente a una aplicación que ha solicitado una reserva se debería clasificar y ser reconocido por los routers en el camino.
- **Políticas**, es necesario tomar acciones cuando las aplicaciones excedan la utilización de los recursos acordados.
- **Encolamiento**, es importante que los dispositivos puedan almacenar los paquetes mientras se envían los que estaban primero.
- **Programación**, funciona junto con el encolamiento y hace referencia al caso en el que existan varias colas y qué cantidad de datos podrían transmitir cada una en cada ciclo.

Los beneficios de Servicios Integrados son el control de admisión de recursos de extremo a extremo, políticas de control de admisión por petición y señalización de números de puerto dinámicos. Como desventajas mencionar que cada flujo activo necesita señalización continua, usando así recursos extra y haciendo que no sea un modelo altamente escalable.

Modelo de servicios diferenciados

Éste modelo es el más actual de los tres y ha sido desarrollado para suplir las deficiencias de sus predecesores. Está explicado detalladamente en las RFC 2474 y 2475. Será el modelo de QoS que se trate con mayor detalle en este libro.

Servicios Diferenciados usa **PHB** (Per-Hop Behavior), que hace referencia al comportamiento por salto. Esto significa que cada salto en el camino está programado para proporcionar un nivel de servicio específico a cada clase de tráfico.

Con este modelo, el tráfico es en principio clasificado y marcado. A medida que fluye en la red va recibiendo distinto trato dependiendo de su marca.

En los servicios diferenciados hay que tener en cuenta que:

- El tráfico es clasificado.
- Las políticas de QoS son aplicadas dependiendo de la clase.
- Se debe elegir el nivel de servicio para cada tipo de clase que corresponderá a unas necesidades determinadas.

Como ventajas principales mencionar la escalabilidad y habilidad para soportar muchos tipos de niveles de servicio. Como puntos negativos el servicio no es absolutamente garantizado y es más complejo de implementar.

MÉTODOS DE IMPLEMENTACIÓN DE QoS

Método antiguo de línea de comandos

CLI (Legacy Command-Line Interface) consiste en la configuración mayormente a nivel de interfaz de los servicios requeridos, haciendo que cada nueva configuración deba implementarse desde cero. Se trata de un modelo de configuración no modular, lo que quiere decir que no ofrece facilidades para aprovechar configuraciones existentes y que induce más al error.

Interfaz de línea de comandos de QoS modular

Cisco ha introducido **MQC** (Modular QoS Command-Line Interface) para resolver las limitaciones del método anterior, además de incluir la utilización de nuevas herramientas de QoS. Se trata de una arquitectura modular, eficiente y más rápida de usar que el método antiguo.

Implementar QoS con MQC requiere de tres pasos necesarios:

- Definir las clases de servicio usando el comando **class-map**.
- Definir políticas para las clases definidas usando el comando **policy-map**.
- Aplicar las políticas en dirección de entrada o salida a una interfaz, usando el comando **service-policy**.

A cada *class map* se le aplica un nombre, sensitivo a mayúsculas y minúsculas, y está compuesta por una o más sentencias. Una o todas esas sentencias deben coincidir dependiendo de si el *class map* tiene el parámetro *match-any* o *match-all*, y en caso de no estar definido por defecto está *match-all*.

El ejemplo que sigue muestra dos *class maps*. La primera se llama VoIP y especifica que todo el tráfico que pase por la ACL 100 será clasificado como VoIP. La segunda es llamada Aplicaciones y especifica que todo el tráfico que pase por la ACL 101 será clasificado como aplicaciones de la empresa.

```
class-map VoIP
match access-group 100
!
class-map Aplicaciones
match access-group 101
```

En el ejemplo ambos *class map* tienen una sola sentencia y como no se ha especificado *match-any* o *match-all* aplica el de por defecto que es *match-all*, aunque al tener una sola sentencia el resultado sería el mismo independientemente de cuál se aplique. También se podría utilizar la condición *no match*.

En la siguiente sintaxis se asocian las características de QoS requeridas a cada clase. Un *policy map* tiene un nombre sensitivo a mayúsculas y minúsculas y permite tener asociadas hasta 256 clases. Se observa como la clase **VoIP** es asignada a una cola de prioridad estricta con un ancho de banda garantizado de 256 Kbs. La clase **Aplicaciones** ha sido asignada a una cola **WFQ** en la que contará con un mínimo de ancho de banda de 256 Kbs. Todo el tráfico restante será asignado a la clase *class-default* y le será dado el ancho de banda restante.

```
policy-map Politicas
class VoIP
priority 256
class Aplicaciones
bandwidth 256
class class-default
fair-queue
```

Finalmente se aplica el policy map a una interfaz, pudiendo aplicarlo a más de una, y dependiendo de la configuración será de entrada o salida. En el ejemplo siguiente se aplica a la interfaz serial 1/0 en dirección de salida:

```
interface serial 1/0
service-policy output Enterprise-Policy
```

Los siguientes tres comandos ayudan a mostrar y verificar las clases y políticas configuradas así como las aplicadas a una interfaz:

- Show class-map
- Show policy-map
- Show policy-map interface interface

AutoQoS

AutoQoS permite generar los comandos de configuración de QoS de un dispositivo de forma automática.

Actualmente se utiliza **AutoQoS Discovery**, capaz de analizar el tráfico de la red para posteriormente generar clases de tráfico basadas en ese análisis. Además según el tipo de interfaz también podría añadir características como fragmentación e *interleaving*, *multilink*, *traffic shaping*, etc.

AutoQoS tiene como principal ventaja que es fácil de implementar y puede hacerse por parte de administradores sin experiencia en QoS. AutoQoS contempla todos estos aspectos a la hora de hacer el despliegue:

- **Clasificación.** *AutoQoS for the Enterprise*, mediante el uso de **AutoQoS Discovery** descubre de forma automática aplicaciones y protocolos, usando **NBAR** (Network Based Application Recognition). Se sirve de CDP para saber si hay teléfonos IP presentes.
- **Generación de políticas.** Proporciona el tratamiento adecuado al tráfico previamente clasificado. También contempla opciones como fragmentación, compresión, etc.
- **Configuración.** Simplemente se necesita introducir el comando **auto qos** en la interfaz elegida.
- **Monitorización y reporte.** Genera mensajes logging, traps de SNMP y resúmenes.

- **Consistencia.** Existe consistencia e interoperabilidad entre diferentes routers configurados con **auto qos**.

Antes de poder usar AutoQoS hay que tener la seguridad de tener CEF habilitado (por defecto lo está) y que cada interfaz posea el ancho de banda correcto, especialmente las interfaces WAN, y configurado el ancho de banda que realmente existe y no el que viene por defecto.

El siguiente ejemplo muestra una interfaz serial configurada con el ancho de banda adecuado, CEF y AutoQoS.

```
ip cef
interface serial 1/0
bandwidth 256
ip address 10.1.1.1 255.255.255.252
auto qos voip
```

En el ejemplo se usa el comando **auto qos voip**, pertenece a la primera generación de AutoQoS. Para usar AutoQoS for Enterprise primero se debe utilizar el comando **auto discovery qos** para analizar y descubrir el tráfico, y posteriormente el comando **auto qos** para terminar la configuración.

Security Device Manager

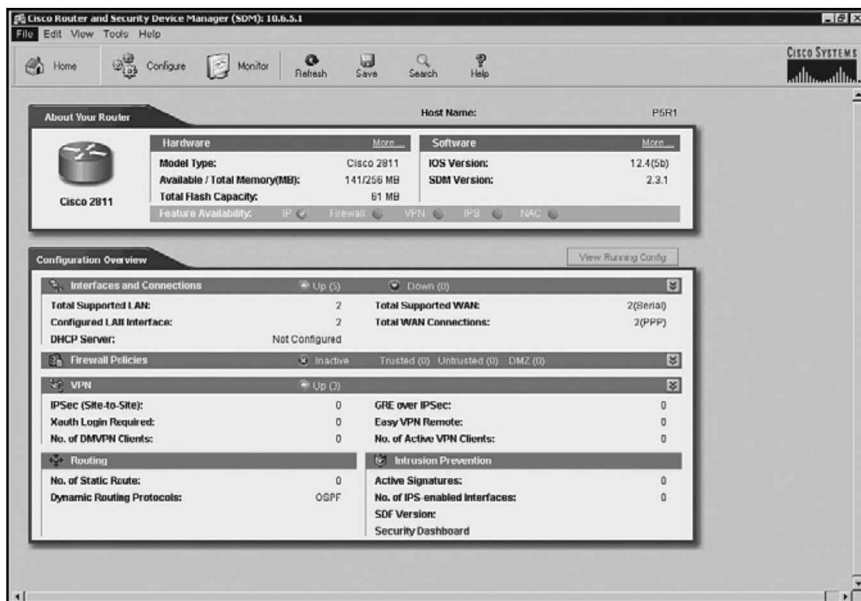
SDM es una aplicación Web que permite realizar diferentes tipos de gestiones sobre varios routers de entre las plataformas 830 a 7301.

El asistente SDM para QoS permite realizar estas tres funciones específicas:

- Implementar QoS
- Monitorizar QoS
- Resolver problemas de QoS

La figura muestra la página principal del SDM, que se compone de dos secciones:

- About Your Router
- Configuration Overview



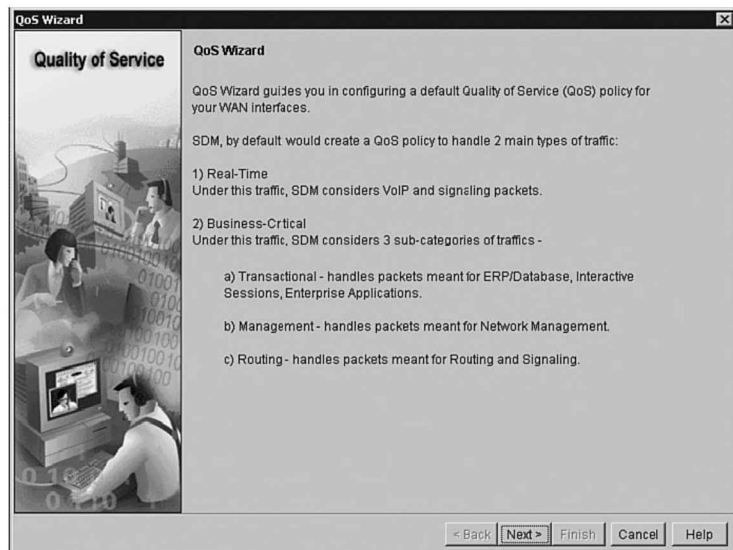
Los otros dos botones importantes en la barra de tareas principal son **Configure** y **Monitor**. Ambos a su izquierda tienen una serie de opciones de entre las cuales se puede configurar y monitorizar calidad de servicio.

Para empezar a crear una política de QoS se deben seguir los siguientes pasos:

- Hacer clic en el botón **Configure** de la barra de herramientas principal del SDM.
- Hacer clic en el botón **Quality of Service** en la barra de herramientas de la izquierda.
- Hacer clic en la pestaña **Create QoS Policy**.
- Hacer clic en **Launch QoS Wizard**.



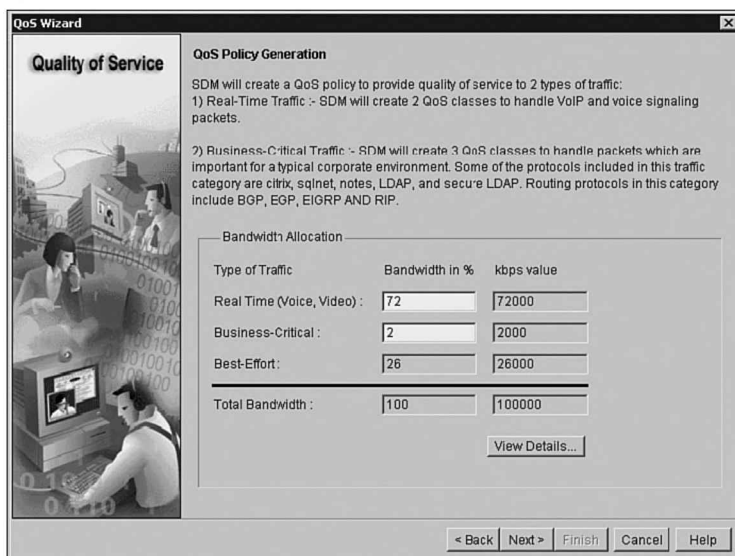
El asistente indica que SDM por defecto creará políticas de QoS para manejar dos tipos de tráfico, Real-Time y Business-Critical.



La siguiente ventana nos pide seleccionar una interfaz a la que se le aplicará la política.



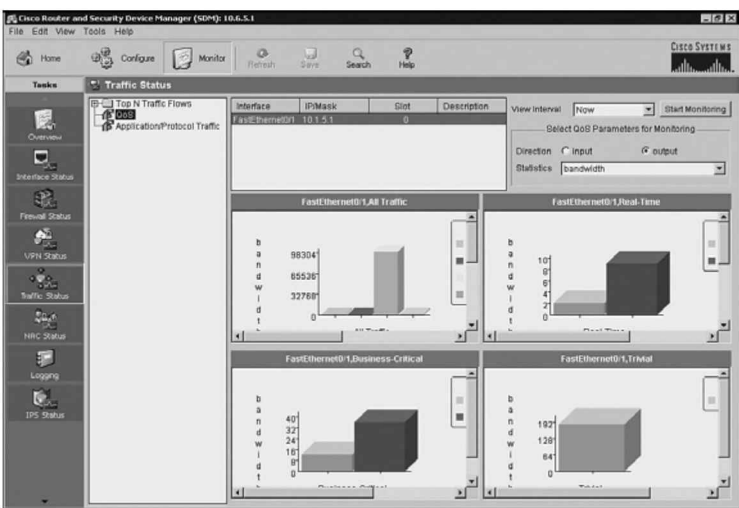
Después solicita introducir los porcentajes de ancho de banda para las dos clases de tráfico creadas.



Por último aparecerá un resumen con todo lo que se ha hecho. Presionando **Finish** se generará la configuración y se aplicará.



Para monitorizar QoS usando SDM hay que hacer clic en el botón **Monitor** de la barra de herramientas de la ventana principal de SDM. Después seleccionar **Traffic Status**.



ADMINISTRACIÓN DEL TRÁFICO

CLASIFICACIÓN Y MARCADO DE TRÁFICO

Clasificar es el proceso de identificar y categorizar tipos de tráfico en clases. La categorización se ha hecho tradicionalmente basándose en ACL, pero además es posible utilizar descriptores de tráfico como:

- Interfaz de entrada.
- Valor del CoS.
- Dirección IP de origen o destino.
- Valor de IP Precedence o DSCP en la cabecera IP.
- Valor EXP en la cabecera MPLS.
- Tipo de aplicación.

La clasificación comienza con el comando **class-map**, que es un componente del Cisco IOS **MQC** (Modular QoS Command-line interface). El parámetro match dentro del class-map puede referirse a una ACL, a un descriptor de tráfico o al protocolo NBAR (Network Based Application Recognition).

Siempre se debe intentar clasificar y marcar el tráfico tan cerca del origen como sea posible. Siendo la capa de acceso de la red el lugar ideal.

Marcar es el proceso de etiquetar tráfico basándose en su categoría. Los campos usados para el marcado dependen de si es en capa 2 (CoS, EXP, DE, CLP) o capa 3 (IP Precedence, DSCP).

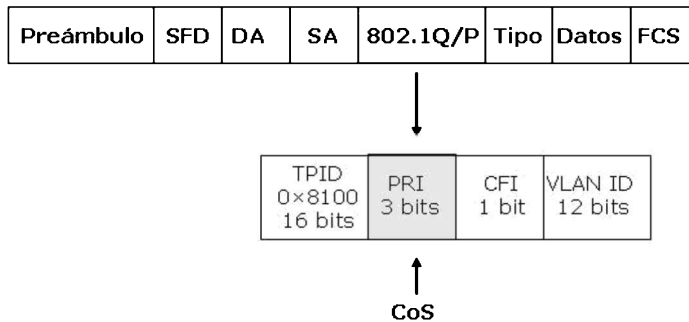
CoS en la trama Ethernet 802.1Q/P

802.1q es un modelo de enlace troncal estandarizado definido por la IEEE. Dentro de la cabecera existe un campo de 3 bits llamado PRI o CoS 802.1p. El **CoS** (Class of Service) es utilizado para propósitos de QoS y puede tener 8 posibles valores.

La siguiente tabla describe los valores de los bits CoS 802.1p dentro de la cabecera:

CoS (bits)	CoS (Decimal)	IETF RFC791	Aplicación
000	0	Routine	Datos
001	1	Priority	Datos de media prioridad
010	2	Immediate	Datos de alta prioridad
011	3	Flash	Señal de llamada
100	4	Flash-Override	Videoconferencia
101	5	Critical	Voz
110	6	Internet	Reservado (inter-network control)
111	7	Network	Reservado(network control)

La siguiente figura muestra la cabecera de 4 bytes 802.1q:



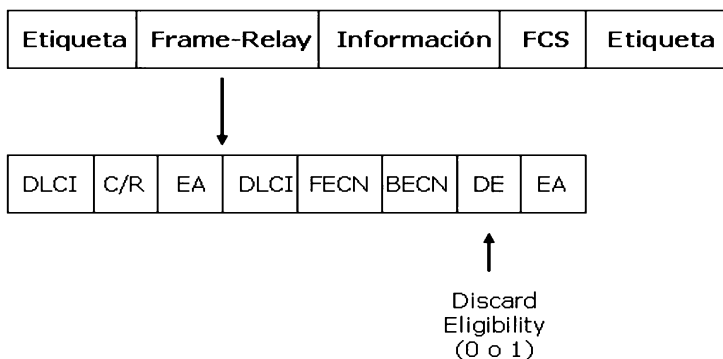
QoS en Frame-Relay y ATM

Los estándares de QoS de Frame Relay y ATM fueron definidos y usados antes de que la IETF definiera los estándares QoS actuales.

En Frame-Relay por ejemplo se han usado los campos **FECN** (Forward Explicit Congestion Notification), **BECN** (Backguard Explicit Congestion Notification) y **DE** (Discard Eligible) de la cabecera para llevar a cabo tareas de notificación de congestión y notificación de probabilidad de descarte.

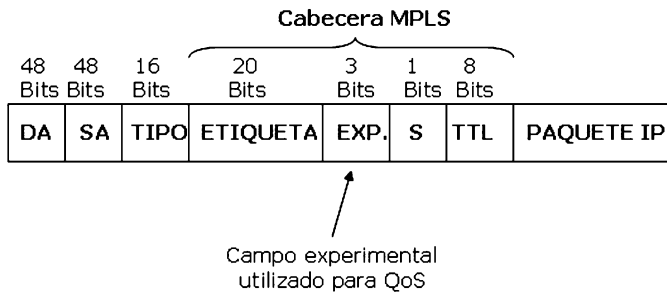
Una celda ATM tiene un bit **CLP** (Cell Lost Priority) para indicar la probabilidad de descarte, si va puesto a 0 tendrá menos probabilidad de ser descartado que si va puesto a 1.

La siguiente figura muestra el campo **DE** en la cabecera Frame-Relay:



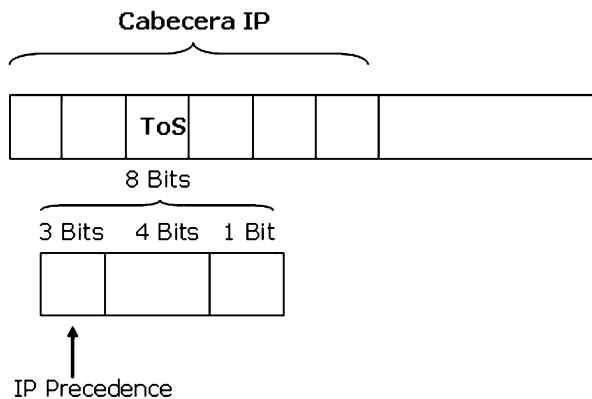
QoS en MPLS

Los paquetes MPLS cuentan con una o más etiquetas de 4 bytes. Cada etiqueta MPLS contiene el campo EXP, que es usado para propósitos de QoS. Dicho campo se compone de 3 bits y por lo tanto es compatible con IP Precedence y CoS.



Originalmente se usaban los 3 bits más significativos (más a la izquierda) del campo TOS, los cuales reciben el nombre de **IP Precedence**, y dan un total de 8 posibles combinaciones, cuanto más alto el número mayor prioridad.

La siguiente figura muestra una cabecera IP y detalla el campo TOS mostrando los posibles valores de IP Precedence.



IP Precedence		
Decimal	Binario	Nombre
0	000	Routine
1	001	Priority
2	010	Immediate
3	011	Flash
4	100	Flash-Override
5	101	Critical
6	110	Internetwork Control
7	111	Network Control

Los valores 6 y 7 son usados por diferentes protocolos en su tráfico de gestión y no está permitido que se configuren para aplicaciones.

La redefinición del byte TOS dio lugar al campo **DiffServ**, usando los 6 bits más significativos (más a la izquierda), lo que aumenta la flexibilidad y las opciones. Los 2 bits menos significativos son llamadas **ECN** (Explicit Congestion Notification) y se usan para control del flujo. **DSCP** (Differentiated Services Code Point) es compatible con IP Precedence, lo que hace que la migración sea más fácil.

En la terminología Diffserv, el comportamiento de reenvío asignado a un DSCP se denomina **PHB** (Per-hop Behavior). El PHB define la precedencia de reenvío que un paquete marcado en relación con otro tráfico del sistema con Diffserv. Esta precedencia determina si el sistema con IP QoS o Diffserv reenvía o descarta dicho paquete. Para un paquete reenviado, cada enrutador Diffserv que el paquete encuentra en la ruta hasta su destino aplica el mismo PHB. La excepción ocurre si otro sistema Diffserv cambia el DSCP.

Existen cuatro tipos de PHB con los valores del DSCP:

- **Selector de Clase PHB**, poniendo a 000 los 3 bits menos significativos del DSCP se obtiene compatibilidad con IP Precedence.
- **PHB por defecto**, con los 3 bits más significativos del IP Precedence/DSCP se obtiene un resultado de Best-effort.
- **PHB Assured Forwarding (AF)**, con los 3 bits más significativos del DSCP puestos a 001, 010, 011 o 100 (AF1, AF2, AF3, AF4) se usa para garantizar ancho de banda.
- **PHB Expedite Forwarding (EF)**, con los 3 bits más significativos del DSCP puestos a 101 (el campo DSCP sería 101110 equivalente a 46 en decimal) se usa para proporcionar un servicio de bajo retardo.

La siguiente tabla muestra el campo DiffServ y las posibles configuraciones del DSCP para los distintos PHB:

Bits DSCP						TIPO
-	-	-	0	0	0	Selector de Clase PHB
0	0	0	-	-	0	PHB por defecto
0	0	1	-	-	0	PHB Assured Forwarding
0	1	0	-	-	0	
0	1	1	-	-	0	
1	0	0	-	-	0	
1	0	1	1	1	0	PHB Expedite Forwarding

El PHB EF proporciona un retardo y jitter bajos, ahora bien, el ancho de banda dedicado para EF ha de ser limitado, sino podría consumir el ancho de banda de las demás clases. EF se debe implementar en colas de prioridad estricta para hacer que los datos se envíen lo más rápido posible.

El equivalente en IP Precedence sería 101 y podría ser usado por dispositivos no preparados para usar DSCP, pero serían compatibles entre sí.

El PHB AF proporciona cuatro colas diferentes para cuatro distintas clases de tráfico (AFxy). Para cada cola se hace una reserva de ancho de banda, en caso de que el tráfico exceda ese ancho de banda podría dar lugar a descarte de paquetes. Para prevenir que esto no ocurra se utilizan técnicas como **WRED** dentro de cada una de las colas, indicando al router qué paquetes descartar en caso de que la cola se empiece a llenar. Dentro de la clase AFxy se especifica la probabilidad de descarte del paquete y puede ser mínima, media o alta (01, 10, 11). Hay que destacar que cuanto menor sea el valor mejor, es decir, menos probabilidad de descarte.

Resumiendo, hay dos características dentro del PHB AF:

- Cuatro clases de tráfico son asignadas a cuatro colas, cada una con un ancho de banda mínimo reservado.
- En cada cola se implementa una técnica para evitar congestión descartando los paquetes elegidos antes de que la cola se llene.

Clase	Probabilidad de descarte		
	Baja	Media	Alta
Clase 1	AF11 DSCP 10: (001010)	AF12 DSCP 12: (001100)	AF13 DSCP 14: (001110)
Clase 2	AF21 DSCP 18: (010010)	AF22 DSCP 20: (010100)	AF23 DSCP 22: (010110)
Clase 3	AF31 DSCP 26: (011010)	AF32 DSCP 28: (011100)	AF33 DSCP 30: (011110)
Clase 4	AF41 DSCP 34: (100010)	AF42 DSCP 36: (100100)	AF43 DSCP 38: (100110)

Es importante tener en cuenta los siguientes puntos acerca de AF:

- Existen cuatro clases: AF1, AF2, AF3 y AF4. Ninguna tiene ventaja sobre las otras, dependerá del modo de configuración.
- Si se utiliza el modelo DSCP, el segundo dígito (y) representa la probabilidad de descarte y es mejor cuanto más bajo, por ejemplo, AF41 tiene menos probabilidad de ser descartado que AF43.
- Es posible encontrar el valor decimal correspondiente para cada AFxy usando la siguiente fórmula:

$$DSCP(decimal) = 8x + 2y$$

Por ejemplo:

$$AF23 = (8 \times 2) + (2 \times 3) = 22$$

- Cada clase AFx es compatible con un valor de IP Precedence. Así AF1y corresponde con IP Precedence 1, AF2y con IP Precedence 2, AF3y con IP Precedence 3 y AF4y con IP Precedence 4.
- Durante la implementación hay que reservar el ancho de banda adecuado a cada cola para evitar retraso y descartes. Es posible utilizar herramientas de QoS que limiten el tráfico que va entrando en cada cola para poder llevarlo a cabo.
- Si existe ancho de banda disponible y la cola AF no está limitada, podría usarlo sobrepasando el límite de lo que tenía reservado.

Hay que tener en cuenta que para poder desplegar QoS de extremo a extremo todos los dispositivos intermedios han de estar configurados y soportar QoS. Esto no es difícil ya que tanto CoS como EXP, IP Precedence, DSCP, etc. pueden ser compatibles entre sí usando los 3 bits de la primera especificación de QoS.

CLASES DE SERVICIO QoS

Planificar e implementar una política de QoS involucra 3 pasos:

- **Paso 1.** Identificar los diferentes tipos de tráfico en la red y sus requerimientos. Es posible la utilización de herramientas como NBAR. Es importante categorizar y documentar los tipos de tráfico según su importancia.
- **Paso 2.** Dividir los diferentes tipos de tráfico previamente identificados en clases. Cada clase de servicio estará compuesta por uno o más tipos de tráfico (recibe el nombre de BA) y tendrá un tratamiento diferente. Como ejemplo el siguiente modelo típicamente usado en ISP:
 - Misión crítica.
 - Tráfico transaccional.
 - Best-effort.
 - Tráfico residual.
- **Paso 3.** Definición de las políticas de QoS para cada clase de servicio. Esto requiere un entendimiento de las necesidades de QoS y de las aplicaciones que corren en la red. Como norma general se suelen usar cuatro o cinco clases de tráfico y lo ideal es no asignar demasiadas aplicaciones a las clases de mayor prioridad. Las siguientes son algunas de las clases de tráfico más comunes:
 - VoIP.
 - Aplicaciones de misión crítica, por ejemplo, SAP y Oracle.
 - Aplicaciones de transacción o interactivas, por ejemplo, Telnet o ssh.
 - Aplicaciones para mover datos, como son FTP o TFTP.
 - Aplicaciones Best-Effort como por ejemplo www.
 - Aplicaciones residuales como por ejemplo P2P.

Una tarea requerida durante la implementación de QoS en la red será unir y traducir entre las marcas de CoS, IP Precedence, DSCP y EXP. La tabla siguiente muestra la manera recomendada por Cisco así como su correspondiente tipo de tráfico:

Cisco AutoQoS	CoS o IP Precedence	DSCP Decimal	DSCP Binario	Nombre
Best Effort	0	0	000000	BE (Best Effort)
Scavenger	1	8	001000	CS1 (Class Selector 1)
Bulk Data	1	10 12 14	001010 001100 001110	AF11 AF12 AF13
Network Management	2	16	010000	CS2 (Class Selector 2)
Telephony Signaling	3	26	011010	AF31
Local Mission Critical	3	28 30	011100 011110	AF32 AF33
Streaming Media Traffic	4	32	100000	CS4 (Class Selector 4)
Interactive Video Traffic	4	34 36 38	100010 100100 100110	AF41 AF42 AF43
Interactive Voice Bearer Traffic	5	46	101110	EF

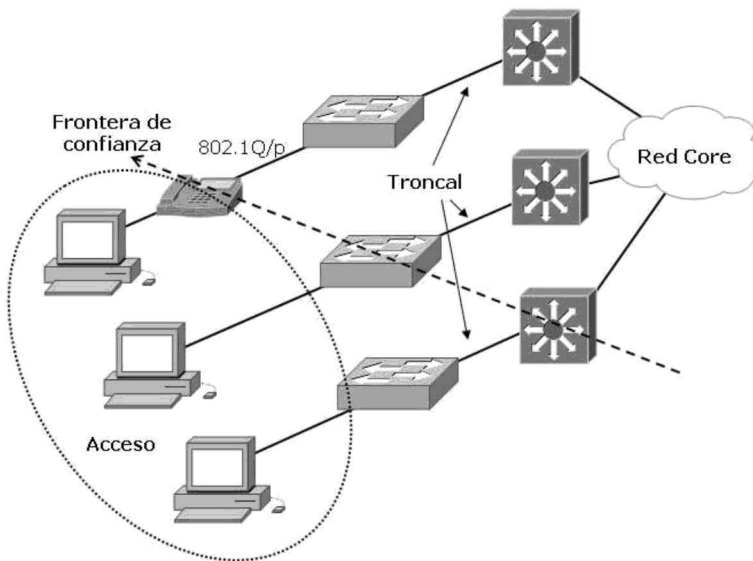
FRONTERAS DE CONFIANZA

Los dispositivos finales como pueden ser PC, teléfonos IP, switches y routers localizados en diferentes niveles de la jerarquía de la red podrían marcar los paquetes IP o las tramas 802.1Q/P. Una medida importante en el diseño es decidir dónde localizar las fronteras de confianza. Dichas fronteras formarán un perímetro, dentro del cual los diferentes dispositivos respetarán y confiarán en las marcas de QoS realizadas dentro de ese perímetro. Las marcas hechas por dispositivos fuera de ese perímetro son eliminadas o chequeadas.

A la hora de decidir dónde colocar la frontera de confianza hay que tener en cuenta que los dispositivos confiables deberían estar dentro de nuestro control administrativo y que dependiendo del dispositivo tendrá capacidad para realizar

unas tareas u otras. Con esto en consideración la frontera de confianza puede ser implementada en una de las siguientes capas:

- Sistema final.
- Capa de Acceso.
- Capa de Distribución.



NETWORK BASED APPLICATION RECOGNITION

NBAR es una potente herramienta de QoS que permite realizar las siguientes funciones:

- Descubrimiento de protocolos. Reconoce un número limitado de protocolos, pero dicho número puede ampliarse cargando nuevos **PDLM** (Packet Description Language Modules) en la memoria flash. Dichos PDLM son publicados por Cisco de manera eventual.
- Recolección de estadísticas de tráfico. Permite identificar aplicaciones y protocolos que están transitando una interfaz en particular, ya sea en dirección de entrada o de salida.
- Clasificación de tráfico. Puede clasificar tráfico inspeccionando el flujo de tráfico más allá de capa 3 o capa 4.

A pesar de tener muchas ventajas, NBAR tiene ciertas limitaciones:

- No soporta interfaces Fast Etherchannel.
- Sólo puede manejar hasta 24 URL, host, o MIMES de manera simultánea.
- Solamente analiza los primeros 400 bytes de cada paquete.
- Sólo funciona si CEF está habilitado.
- No soporta paquetes multicast, fragmentados y los asociados con tráfico HTTP seguro.
- No puede analizar tráfico destinado u originado en el router en el que NBAR está configurado, debido a que dicho tráfico no es procesado por CEF.

CONFIGURACIÓN DE NBAR

Como primera medida es necesario verificar si es preciso ampliar la lista de protocolos y aplicaciones que NBAR es capaz de reconocer. Será necesario descargar del website de Cisco diferentes PDLM, para lo que será imprescindible el acceso con cuenta CCO.

Una vez efectuada la descarga se debe copiar en la flash del router o en un servidor TFTP. El siguiente comando permite instalar el archivo:

```
Router(config)#ip nbar pdlm pdln-name
```

El formato del **pdln-name** ha de ser tipo URL, por ejemplo, para instalar el archivo desde la flash se utiliza el comando **flash://cisco.pdlm**; mientras que para instalarlo desde el servidor TFTP, **tftp://10.0.0.1/cisco.pdlm**.

Para modificar el número de puerto asociado a un protocolo o añadir un puerto a la lista de puertos asociada con un protocolo se utiliza el siguiente comando:

```
Router(config)#ip nbar port-map protocol-name [tcp | udp] port-number
```

Es posible especificar hasta 16 números de puerto adicionales.

Para ver la relación actual entre protocolos y puertos se utiliza el siguiente comando:

```
Router#show ip nbar port-map [protocol-name]
```

El siguiente ejemplo muestra la salida del comando anterior.

```
Router# show ip nbar portmap
port-map bgp tcp 179
port-map dhcp udp 67 68
port-map dns udp 53
port-map dns tcp 53
.....
```

Para habilitar el descubrimiento de protocolos de NBAR se utiliza el siguiente comando:

```
Router(config-if)#ip nbar protocol-discovery
```

Para mostrar las estadísticas recopiladas se utiliza el siguiente comando, si no se especifica una interfaz mostrará las estadísticas de todas.

```
Router#show ip nbar protocol-discovery
```

El ejemplo muestra un ejemplo de la salida de este comando.

```
Router# show ip nbar protocol-discovery
Ethernet 0/0/0
```

	Input	Output
Protocol	Packet Count	Packet Count
	Byte Count	Byte Count
	5 minute(bps)	5 minute(bps)
eigrp	60	0
	3600	0
	0	0
bgp	0	0
	0	0
	0	0

Al usar NBAR para clasificar un tipo de tráfico específico es posible utilizarlo dentro de un **class map**. Para ello se utiliza el modelo de configuración MQC:

```
Router(config-cmap)#match protocol protocol-name
```

Donde **protocol-name** se refiere a una aplicación o protocolo soportada por NBAR tal como puede ser www, ip, ftp, ipx, etc.

El siguiente ejemplo muestra cómo usar NBAR para identificar tráfico HTTP (clasificarlo en un class map), para posteriormente limitarlo con un policy map y finalmente aplicarlo a una interfaz. Se ha modificado el protocolo para que además del puerto 80 tenga en cuenta el 8080 típicamente usado por proxy. Lo que resume el modelo MQC de configuración de QoS. Finalmente se ha configurado NBAR en la interfaz serial 1/1.

```
ip nbar port-map http tcp 80 8080
!
class-map www
  match protocol http
!
policy-map www-ltd-bw
  class www
    bandwidth 512
!
interface serial 1/1
  ip nbar protocol-discovery
  service-policy output www-ltd-bw
!
```

Una de las características más importantes de NBAR es su capacidad de realizar una inspección de los paquetes más allá de capa 3 o 4. La clasificación del tráfico se realiza según los siguientes criterios:

- Clasificación del tráfico basándose en el nombre de la URL o parte de ella.
- Clasificación del tráfico basándose en el tipo MIME.
- Clasificación del tráfico de transferencia de archivos entre aplicaciones punto a punto usando expresiones regulares.
- Clasificación del tráfico basándose en los tipos de datos RTP o en el códec.

Los comandos utilizados dentro del class map para categorizar tráfico acorde a las clasificaciones vistas son los siguientes:

```
Router(config-cmap)#match protocol http url url-string
Router(config-cmap)#match protocol http host host-name
Router(config-cmap)#match protocol http mime mime-type
Router(config-cmap)#match protocol fasttrack file-transfer regular-expression
Router(config-cmap)#match protocol rtp [audio | video |payload-type payload-type-string]
```

ADMINISTRACIÓN DE COLAS Y CONGESTIÓN

CONGESTIÓN Y COLAS

La congestión ocurre cuando el ritmo con el que llegan los paquetes al router es mayor que el ritmo con el que salen. Esto puede ser causado principalmente cuando la o las interfaces de salida tiene menos capacidad o son más lentas que la interfaz de entrada, como por ejemplo si los paquetes llegan en un enlace de Giga y han de salir por un enlace Ethernet. También podría ocurrir que el tráfico llegase por dos o más interfaces y solamente pudiera salir por una, que tiene menos ancho de banda que las de entrada.

Los dispositivos de red pueden reaccionar de diferentes maneras ante una congestión. Para situaciones en que la congestión sea permanente habría que pensar en un incremento del ancho de banda, pero para situaciones donde la congestión es temporal es posible implementar diferentes técnicas de encolamiento, que se pueden elegir dependiendo del objetivo que se busque. Si una cola está llena y le llegan nuevos paquetes, se produce un fenómeno conocido como **tail drop**. Dicho fenómeno hace que mientras la cola esté llena los paquetes entrantes sean descartados a medida que llegan. Para evitar la caída o el tail drop ciertos paquetes que están en la cola se descartan para evitar que todos los nuevos paquetes entrantes sean descartados. La elección de qué paquetes serán descartados dependerá del tipo de cola. Por defecto todas las interfaces, excepto aquellas con una velocidad de 2,048 Mbps o menos, usan un sistema **FIFO** (First In, First Out).

La arquitectura de encolamiento en una interfaz está compuesta por dos componentes, la cola de software y la cola de hardware (TxQ). Si la cola de hardware no se llena, no habrá paquetes en la cola de software. Pero si la cola de hardware estuviera congestionada o llena, entonces los paquetes se almacenarían en la cola de software y serían procesados por el mecanismo de encolamiento que hubiera implementado (FIFO, PQ, CQ, RR, LLQ, etc.) para posteriormente ser pasados a la cola de hardware, que siempre usa un mecanismo FIFO.

El mecanismo de encolamiento por software normalmente cuenta con un determinado número de colas, cada una perteneciente a una clase de tráfico, donde los paquetes son asignados una vez que llegan a la interfaz. En caso de que la cola esté llena los paquetes son descartados directamente (tail drop).

Si no existieran las colas de software todo el tráfico sería encolado a través de la cola de hardware que siempre es FIFO, de manera que ciertas aplicaciones sufrirían más que otras y no habría manera de dar un trato adecuado al tráfico. En caso de configurar la cola de hardware con un valor demasiado grande se obtendría un resultado similar al mencionado anteriormente; mientras que si la cola de hardware es demasiado pequeña se llenará pronto y los nuevos paquetes serán asignados más rápidamente a la cola de software, que dependiendo de su configuración les dará un trato u otro.

Se recomienda no cambiar la configuración de la cola de hardware a no ser que sea necesario, el comando **tx-ring-limit** modifica el modo de configuración de la cola en la interfaz.

Podemos ver el tamaño de la cola de hardware con el comando **show controllers**.

```
Router#show controllers
Interface EOBC0
Hardware is DEC21143
dec21140_ds=0x6107CA20, registers=0x3C018000, ib=0x78A7380
rx ring entries=128, tx ring entries=256, af setup failed=0
rxring=0x78A7480, rxr shadow=0x6107CC0C, rx_head=6, rx_tail=0
txring=0x78A7CC0, txr shadow=0x6107CE38, tx_head=18, tx_tail=18,
tx_count=0
PHY link up
CSR0=0xF8024882, CSR1=0xFFFFFFFF, CSR2=0xFFFFFFFF, CSR3=0x78A7480
CSR4=0x78A7CC0, CSR5=0xF0660000, CSR6=0x320CA002, CSR7=0xF3FFA261
CSR8=0xE0000000, CSR9=0xFFFD3FF, CSR10=0xFFFFFFFF, CSR11=0x0
CSR12=0xC6, CSR13=0xFFFF0000, CSR14=0xFFFFFFFF, CSR15=0x8FF80000
DEC21143 PCI registers:
bus_no=0, device_no=6
CFID=0x00191011, CFCS=0x02800006, CFRV=0x02000041, CFLT=0x0000FF00
CBIO=0x01124401, CBMA=0x48018000, CFIT=0x28140100, CFDD=0x00000400
```

```

MII registers:
Register 0x00:  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF
Register 0x08:  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF
Register 0x10:  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF
Register 0x18:  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF  FFFF
throttled=0, enabled=0, disabled=0
rx_fifo_overflow=0, rx_no_enp=0, rx_discard=0
tx_underrun_err=0, tx_jabber_timeout=0, tx_carrier_loss=0
tx_no_carrier=0, tx_late_collision=0, tx_excess_coll=0
tx_collision_cnt=26, tx_deferred=0, fatal_tx_err=0, tbl_overflow=0
HW addr filter: 0x78D10E0, ISL Disabled
Entry= 0:  Addr=0000.0000.0000
Entry= 1:  Addr=0000.0000.0000
Entry= 2:  Addr=0000.0000.0000
Entry= 3:  Addr=0000.0000.0000
Entry= 4:  Addr=0000.0000.0000
Entry= 5:  Addr=0000.0000.0000
Entry= 6:  Addr=0000.0000.0000
Entry= 7:  Addr=0000.0000.0000

```

First In First Out

FIFO (First In First Out) es el mecanismo con que la cola de hardware siempre procesa los paquetes. Las colas de software se utiliza por defecto en interfaces de velocidad igual a 2,048 Mbps o inferior, como pueden ser las E1. La configuración no es compleja, un sólo comando basta para habilitarla, y los paquetes simplemente llegan, se ponen en la cola y esperan a ser enviados, el primero en llegar será el primero en salir. La clase de paquete o la prioridad no importan, simplemente importa quién llega primero. Esto puede ocasionar un problema debido a que algunas aplicaciones de descarga como FTP pueden saturar en enlace evitando el paso de paquetes de VoIP o dejando pasar solamente algunos, causando que las llamadas suenen entrecortadas o que incluso se caigan.

Priority Queuing

PQ (Priority Queuing) tiene cuatro colas de prioridad disponibles:

1. Alta
2. Media
3. Normal
4. Baja

Es necesario determinar qué paquetes estarán en cada una de ellas, si no se configuran serán asignadas por defecto a la normal. Como mecanismo para clasificar paquetes dentro de las colas normalmente se usan ACL. Siempre que haya paquetes dentro de la cola de prioridad alta serán enviados hasta que esté vacía, entonces, el algoritmo enviará un paquete de la cola de prioridad media, para después volver a la cola de prioridad alta; en caso de que no tenga nada volverá a la cola de prioridad media, y en caso de que ésta tampoco tenga nada irá a la cola de prioridad normal para enviar un sólo paquete y volver a la cola de prioridad alta, y así sucesivamente. PQ siempre prioriza el tráfico de la cola de prioridad alta, no dando lugar a enviar paquetes en las otras hasta que esté vacía, por lo tanto hay que configurar este tipo de cola con precaución.

Con el comando **priority-list** se puede definir el tráfico que irá destinado dentro de cada una de las colas.

```
Router(config)#access-list 101 permit ip any any precedence 5 tos 12
Router(config)#access-list 102 permit ip any any precedence 4
Router(config)#access-list 103 permit ip any any precedence 3
Router(config)#priority-list 1 protocol ip high list 101
Router(config)#priority-list 1 protocol ip medium list 102
Router(config)#priority-list 1 protocol ip normal list 103
Router(config)#priority-list 1 default low
Router(config)#interface Ethernet0
Router(config-if)#priority-group 1
```

Round Robin

En el método **RR** (Round Robin) existen un conjunto de colas a las que se les asignan distintos tipos de tráfico; el concepto es enviar un paquete de cada una y pasar a la siguiente, así sucesivamente. Se empieza desde la primera, se envía un paquete y se pasa a la segunda, a la tercera, hasta llegar a la última y empezar de nuevo. Es un método equitativo pero si una de las colas tuviera paquetes de mayor tamaño que las otras estaría disfrutando de más prioridad, al estar enviando más cantidad de datos en cada turno. No es posible priorizar tráfico con este método debido a que todas las colas son tratadas igualmente.

Weighted Round Robin

WRR (Weighted Round Robin) es una versión modificada de RR en la que se pueden asignar pesos a las colas, haciendo que esos valores correspondan con el ancho de banda que tienen permitido utilizar, de esta manera se pueden favorecer a determinadas colas para que puedan enviar más cantidad de datos que otras durante su turno.

La configuración de WRR debe ser cuidadosa, se puede incurrir en el error de no estar dividiendo el ancho de banda exactamente como se piensa. Por ejemplo, en una interfaz con una MTU de 1500 bytes (Ethernet por defecto) donde se ha configurado **Custom Queue** (CQ), que es un mecanismo de encolamiento basado en WRR. Se han creado dos colas para procesar 3000 bytes de cada cola por turno, la primera cola envía dos paquetes de 1500 bytes cada uno y pasa el turno a la segunda cola, que tiene esperando tres paquetes, el primero es de 1500 bytes, el segundo es de 1400 y el tercero es de 1500. Al sumar los dos primeros encuentra que todavía hay espacio y añade el tercero, como consecuencia hay un exceso de 1400 bytes durante su turno.

WEIGHTED FAIR QUEUING

WFQ (Weighted Fair Queuing) es el método de encolamiento por defecto en los router Cisco para interfaces serial de 2,048 Mbps (E1) o inferiores y es utilizado por **CBWFQ** y **LLQ**.

WFQ realiza las siguientes funciones:

- Divide el tráfico en flujos.
- Proporciona una cantidad de ancho de banda justo a los flujos activos.
- Hace que los flujos con poco volumen de tráfico sean despachados más rápido.
- Proporciona más ancho de banda a los flujos con más prioridad.

WFQ elimina los problemas de retraso, jitter, sobre utilización de un flujo que posee FIFO y la sobre utilización de la cola de prioridad en el sistema PQ.

WFQ es un algoritmo basado en flujos, los paquetes llegan y son categorizados en diferentes tipos de flujos asignados a diferentes colas FIFO. Los flujos pueden ser identificados según a lo siguiente:

- Dirección IP de origen.
- Dirección IP de destino.
- Número de protocolo.
- Campo ToS.

- Número de puerto de origen TCP/UDP.
- Número de puerto de destino TCP/UDP.

Un cifrado o hash es generado basándose en los valores previos y como todos los paquetes pertenecientes a un determinado flujo tendrán el mismo hash, serán por lo tanto asignados a la misma cola.

WFQ no funciona con interfaces túneles o con encriptación debido a que el algoritmo ha de acceder a los campos de la cabecera del paquete para calcular el hash.

Hay que tener en cuenta que dependiendo de cómo esté de llena la cola de la interfaz, y a su vez de si está dentro del umbral de descarte, el paquete podría ser descartado. A los paquetes que entran en la cola se les asigna un número de secuencia.

En caso de que todos los flujos tengan el mismo peso y el mismo nivel de prioridad, WFQ divide equitativamente el ancho de banda de la interfaz entre los diferentes flujos. Como resultado los flujos con poco volumen de tráfico son enviados rápidamente a la cola de hardware mientras que los flujos con alto volumen de tráfico construyen sus correspondientes colas y esperan a ser enviados.

El número de colas es dinámico y depende del número de flujos, WFQ se encarga de añadir o borrar colas cuando sea necesario. La cantidad de colas está limitada por defecto para permitir hasta 256, pero es posible modificar la configuración y cambiar los valores entre 16 y 4096 (potencia de 2). Los flujos dinámicos WFQ permiten hasta 8 colas para paquetes del sistema y hasta 1000 colas para flujos RSVP. En caso de que se excediera el número de flujos activos se asignarían a colas ya existentes, pero habría que plantearse entonces otro sistema de encolamiento.

WFQ posee una cola de contención para todos los paquetes de todos los flujos y su valor es el resultado de la suma de todos los paquetes presentes en la arquitectura WFQ. Si un paquete llegara cuando la cola de contención está llena sería descartado. Este mecanismo es denominado “descarte agresivo”. Hay una excepción, y es que si el paquete fuera asignado a una cola vacía se evitaría que fuese descartado.

Cada cola dentro de WFQ cuenta con un umbral de descarte denominado Congestive Discard Threshold (**CDT**). Dicho **CDT** se usa para el “descarte temprano” y funciona de la siguiente manera: cuando llega un nuevo paquete y la cola no está llena pero se ha alcanzado el umbral CDT, se comprueba el número de

secuencia del paquete entrante con el de otros paquetes, si se encuentra otro paquete en esa cola u otra con un número de secuencia mayor que el del paquete entrante, se descarta el del número y secuencia más altos. WFQ da números de secuencia altos a paquetes procedentes de flujos de alta demanda de ancho de banda, como por ejemplo uno proveniente de una descarga FTP.

WFQ tiene los siguientes beneficios:

- La configuración es simple y no hace falta clasificar previamente.
- Se garantiza que todas las colas podrán enviar paquetes.
- Se descartan paquetes en flujos de tráfico agresivos y se agiliza los no agresivos.
- Es un protocolo estándar.

WFQ tiene las siguientes desventajas:

- El sistema de clasificación y la programación de la salida de los paquetes de la cola no puede ser modificada.
- Solamente es soportado en enlaces lentos (hasta 2,048 Mbps).
- No se garantiza prevención del retraso o un ancho de banda mínimo para ningún flujo.
- Podría darse el caso de que múltiples flujos fueran asignados a la misma cola.

Configuración y monitorización de WFQ

WFQ está habilitado por defecto en interfaces seriales que tengan un ancho de banda igual o menor a 2,048 Mbps. Para deshabilitarlo y habilitarlo nuevamente se utiliza el comando **fair queue**. Los siguientes comandos muestran los parámetros opcionales WFQ:

```
Router(config-if)#fair-queue [cdt [dynamic-queues [reservable-  
queues]]]  
Router(config-if)#hold-queue max-limit out
```

El segundo comando sirve para configurar el máximo número de paquetes que una interfaz puede almacenar en su cola de software de salida.

El parámetro **cdt** indica el umbral de paquetes permitidos en cada cola antes de comenzar a descartar, por defecto es 64 y puede configurarse con valores de potencia de 2 entre 16 y 4096. El parámetro **dynamic-queues** permite indicar el máximo número de colas dinámicas que se pueden generar en valores entre 16 y 4096, por defecto es 256. El parámetro **reservable-queues** establece el número de colas reservadas, por defecto es 0 y podemos configurarlo con valores entre 0 y 1000. Se usa por ejemplo en interfaces configuradas con RSVP.

Para ver la configuración de WFQ se utiliza el comando **show interface**:

```
Router#show interfaces serial 1/0
Serial1/0 is up, line protocol is up
Hardware is CD2430 in sync mode
MTU 1500 bytes, BW 128000 Kbit, DLY 20000 usec,
reliability 255/255, txload 1/255, rxload 1/255
Encapsulation FRAME-RELAY, loopback not set
Keepalive not set
LMI DLCI 1023 LMI type is CISCO frame relay DTE
FR SVC disabled, LAPF state down
Broadcast queue 0/64, broadcasts sent/dropped 105260/0, interface
broadcasts 9 2894
Last input 00:00:00, output 00:00:02, output hang never
Last clearing of "show interface" counters 2d20h
Input queue: 0/75/0/0 (size/max/drops/flushes); Total output drops:
0
Queueing strategy: weighted fair
Output queue: 0/1000/64/0 (size/max total/threshold/drops)
Conversations 0/10/256 (active/max active/max total)
Reserved Conversations 0/0 (allocated/max allocated)
Available Bandwidth 96000 kilobits/sec
5 minute input rate 2000 bits/sec, 1 packets/sec
5 minute output rate 2000 bits/sec, 0 packets/sec
228008 packets input, 64184886 bytes, 0 no buffer
Received 0 broadcasts, 0 runts, 0 giants, 0 throttles
0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored, 0 abort
218326 packets output, 62389216 bytes, 0 underruns
0 output errors, 0 collisions, 3 interface resets
0 output buffer failures, 0 output buffers swapped out
0 carrier transitions
DCD=up DSR=up DTR=up RTS=up CTS=up
```

El comando **show queue interface** muestra información más detallada:

```
Router# show queue atm2/0.33 vc 33
Interface ATM2/0.33 VC 0/33
Queueing strategy: weighted fair
Total output drops per VC: 18149
Output queue: 57/512/64/18149 (size/max total/threshold/drops)
Conversations 2/2/256 (active/max active/max total)
Reserved Conversations 3/3 (allocated/max allocated)
```

```
(depth/weight/discards/tail drops/interleaves) 29/4096/7908/0/0
Conversation 264, linktype: ip, length: 254
source: 10.1.1.1, destination: 10.0.2.20, id: 0x0000, ttl: 59,
TOS: 0 prot: 17, source port 1, destination port 1
(depth/weight/discards/tail drops/interleaves) 28/4096/10369/0/0
Conversation 265, linktype: ip, length: 254
source: 10.1.1.1, destination: 10.0.2.20, id: 0x0000, ttl: 59,
TOS: 32 prot: 17, source port 1, destination port 2
```

CLASS BASED WEIGHTED FAIR QUEUING

CBWFQ (Class Based Weighted Fair Queuing) permite la definición manual de clases, cada una de las cuales es asignada a su propia cola. Dichas clases se definen mediante el uso de class maps. Cada una de las colas tiene definido un mínimo de ancho de banda que puede utilizar, pero como su propio nombre indica, es un mínimo, en caso de haber más ancho de banda libre podría emplearlo.

CBWFQ permite la creación de hasta 64 colas, cada una de las cuales es del tipo FIFO, con un ancho de banda garantizado y un límite máximo de paquetes, que en caso de ser alcanzado produciría un **tail drop**, aunque podría evitarse con métodos avanzados como **WRED**. Además de estas posibles 64 colas existe una denominada clase por defecto (class-default) que usa un método de encolamiento WFQ y que si no se especifica lo contrario le será asignado el ancho de banda restante, al igual que el resto de las colas permite usar WRED. Existen plataformas como los 7500 que permiten configurar incluso otras 64 colas con WFQ en vez de FIFO.

CBWFQ tiene los siguientes beneficios:

- Permite la definición de clases de tráfico mediante el uso de class maps.
- Permite la reserva de ancho de banda por cada clase de tráfico basándose en ciertos criterios.
- Proporciona granularidad al permitir definir hasta 64 clases diferentes de flujos de tráfico.

La desventaja de CBWFQ es que no incorpora ningún mecanismo para favorecer el tráfico en tiempo real de aplicaciones como VoIP o vídeo.

Clasificación, programación y garantía de ancho de banda

La clasificación del tráfico en CBWFQ se realiza mediante MQC, usando class maps. Las posibilidades de clasificación dependen de la versión de IOS, tipo de interfaz, encapsulación, etc.

La garantía de ancho de banda ofrecida en cada una de las colas está basada en una de estas tres posibles asignaciones:

- **Ancho de banda:** utilizando el comando **bandwidth** se puede reservar una cierta cantidad de ancho de banda (medido en Kbps) para la clase. Este ancho de banda es tomado del disponible en la interfaz. Por defecto el máximo ancho de banda disponible en una interfaz es el 75% del ancho de banda total de la misma, pero es modificable mediante el comando **max-reserved-bandwidth**.
- **Porcentaje de ancho de banda:** utilizando el comando **bandwidth percent** se puede reservar una cantidad de ancho de banda de manera porcentual al ancho de banda disponible en la interfaz. Se puede sobrescribir el ancho de banda de una interfaz con el comando **bandwidth**.
- **Porcentaje del ancho de banda que queda:** utilizando el comando **bandwidth remaining percent** es posible asignar a una clase determinada cierta cantidad del ancho de banda sobrante en la interfaz. Cualquiera que sea esta cantidad será restada de la porción de ancho de banda disponible del ancho de banda total de dicha interfaz.

El comando **max-reserved-bandwidth** se configura a nivel de interfaz, su valor por defecto es 75, lo que significa que el 75% del ancho de banda disponible de la interfaz puede ser reservado.

Para calcular el ancho de banda disponible se multiplica el ancho de banda de la interfaz por el ancho de banda reservado y restando a ese resultado todas las reservas hechas. Por defecto al reservar un 75% del ancho de banda de la interfaz queda un 25% para tráfico de control como puede ser CDP, por ejemplo. Si existe la necesidad de modificar este valor habría que hacerlo conociendo los posibles resultados.

Configuración y monitorización de CBWFQ

El primer paso en la configuración CBWFQ es definir las clases de tráfico mediante el uso de class maps, como muestra la siguiente sintaxis:

```
!  
class-map Transacciones  
match access-group 100  
!  
class-map Aplicaciones  
match access-group 101  
!
```

El segundo paso es crear un policy-map para asociar los class-maps y donde se configurarán las opciones específicas para cada cola, como muestra la siguiente sintaxis:

```
!  
policy-map Politicas-empresariales  
  class Transacciones  
    bandwidth 128  
    queue-limit 50  
  class Aplicaciones  
    bandwidth 256  
    queue-limit 90  
  class class-default  
    fair-queue 16
```

Por último se aplica el policy-map a una interfaz a través del comando **service-policy** a nivel de interfaz:

```
interface serial 1/0  
service-policy output Politicas-empresariales
```

Para ver la información relativa a un policy-map se utiliza el comando **show policy-map interface**:

```
Router# show policy-map interface e1/1  
Ethernet1/1 output : pol  
Weighted Fair Queueing  
Class class1  
Output Queue: Conversation 264  
Bandwidth 937 (kbps) Max Threshold 64 (packets)  
(total/discards/tail drops) 11548/0/0  
Class class2  
Output Queue: Conversation 265  
Bandwidth 937 (kbps) Max Threshold 64 (packets)  
(total/discards/tail drops) 11546/0/0  
Class class3
```

```
Output Queue: Conversation 266  
Bandwidth 937 (kbps) Max Threshold 64 (packets)  
(total/discards/tail drops) 11546/0/0
```

LOW LATENCY QUEUING

Hasta ahora no se ha tratado ningún mecanismo de encolamiento que cubriera las necesidades que tienen las aplicaciones en tiempo real. **LLQ** (Low Latency Queuing) cuenta con una cola de prioridad estricta que es usada para aplicaciones en tiempo real que son sensitivas al retraso y al jitter. Pero a diferencia de QC por ejemplo, en LLQ esta cola está limitada, impidiendo así que anule a las demás. Para lograr un mecanismo, se le asigna un ancho de banda a la cola, pero limitando su uso. En caso de que haya congestión LLQ sólo usará el ancho de banda que se le ha asignado, permitiendo así que las demás colas también puedan enviar.

Este sistema es similar a CBWFQ pero difiere en la adición de colas de prioridad estricta con el uso del comando **strict-priority**. Se debe tener en cuenta que son posibles más de una cola de prioridad estricta, siendo dos usadas en muchos casos para VoIP y vídeo.

LLQ ofrece todos los beneficios de CBWFQ y además proporciona una o más colas de prioridad estricta que garantizarán ancho de banda a aplicaciones sensitivas al retraso y al jitter. Dichas colas no evitarán que el resto puedan seguir transmitiendo durante períodos de congestión ya que estarán limitadas.

Configuración y monitorización de LLQ

La configuración es casi idéntica a la de CBWFQ excepto por el uso del parámetro **priority** en vez de **bandwidth**.

```
router(config-pmap-c)#priority bandwidth {burst}  
router(config-pmap-c)#priority percent percentage {burst}
```

El parámetro **burst** se refiere a ráfagas temporales y se mide en bytes. Se puede definir un valor entero de entre 32 a 2000000. De esta manera permitirá de manera temporal ráfagas (tráfico por encima del ancho de banda limitado) de hasta la cantidad de bytes configurados.

```
router(config)# policy-map empresarial  
router(config-pmap)# class voz  
router(config-pmap-c)# priority 50  
router(config-pmap)# class negocio  
router(config-pmap-c)# bandwidth 200
```

```
router(config-pmap)# class class-default
router(config-pmap-c)# fair-queue
```

Para monitorizar LLQ se utiliza el comando **show policy-map interface**:

```
router# show policy-map interface serial 1/0
Serial1/0
Service-policy output: AVVID (2022)
Class-map: supercontrol (match-all) (2035/5)
4253851 packets, 306277272 bytes
1 minute offered rate 499000 bps, drop rate 0 bps
Match: ip dscp 46 (2037)
Strict Priority
Output Queue: Conversation 264
Bandwidth 500 (kbps)
(pkts matched/bytes matched) 4248148/305866656
(total drops/bytes drops) 5/360
Class-map: control (match-all) (2023/2)
251162 packets, 375236028 bytes
1 minute offered rate 612000 bps, drop rate 0 bps
Match: ip dscp 18 20 22 (2025)
Weighted Fair Queueing
Output Queue: Conversation 265
Bandwidth 25 (%)
(pkts matched/bytes matched) 3/4482
(depth/total drops/no-buffer drops) 0/0/0
mean queue depth: 0
Dscp Random drop Tail drop Minimum Maximum Mark
(Prec) pkts/bytes pkts/bytes threshold threshold probability
0(0) 0/0 0/0 20 40 1/10
1 0/0 0/0 22 40 1/10
2 0/0 0/0 24 40 1/10
3 0/0 0/0 26 40 1/10
4 0/0 0/0 28 40 1/10
(...up to DSCP 63.....)
61 0/0 0/0 30 40 1/10
62 0/0 0/0 32 40 1/10
63 0/0 0/0 34 40 1/10
rsvp 0/0 0/0 36 40 1/10
.....
Class-map: class-default (match-any) (2039/0)
4719109 packets, 1000522466 bytes
1 minute offered rate 1625000 bps, drop rate 0 bps
Match: any (2041)
4719109 packets, 1000522466 bytes
1 minute rate 1625000 bps
```

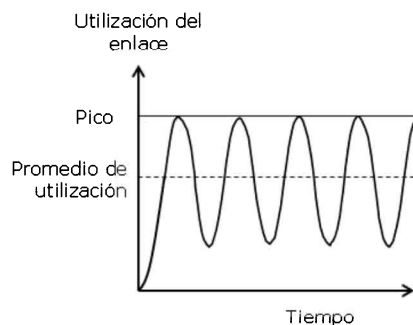
EVITANDO LA CONGESTIÓN

Evitar la congestión engloba una serie de mecanismos de QoS que sirven para que no se produzca el fenómeno **tail drop**, es decir, para evitar en la medida de lo posible que se descarten paquetes. Dichos mecanismos pueden ser RED, WRED y CBWRED.

Limitaciones de tail drop

El fenómeno tail drop ocurriría cuando llegaban nuevos paquetes a una cola llena, la que simplemente los descarta ante la imposibilidad de encolarlos.

Tail drop tiene consecuencias nefastas en flujos de datos TCP. Una de ellas es la sincronización global de TCP. Cuando Tail drop ocurre, los flujos de datos TCP (asumiendo que hay varios) se detienen y reducen su tamaño de ventana simultáneamente, a esto se le conoce como **slow start**. A partir de este proceso la utilización de la interfaz baja notablemente, está menos congestionada y aumenta el ancho de banda disponible. Siguiendo el comportamiento natural de TCP, los flujos de datos empiezan a aumentar el tamaño de ventana. En un período de tiempo la interfaz volverá a estar congestionada y el ciclo se repetirá de nuevo.



El resultado de la sincronización global de TCP es pasar de períodos de congestión a períodos de baja utilización de la interfaz, ambos casos son no deseados.

Este fenómeno solamente afecta a flujos de tráfico TCP y durante este proceso, los flujos de tráfico UDP o tráfico no IP, si los hubiera, tomarían ventaja y utilizando el ancho de banda disponible después de un período de congestión, dejarían a los flujos TCP quizás fuera de juego.

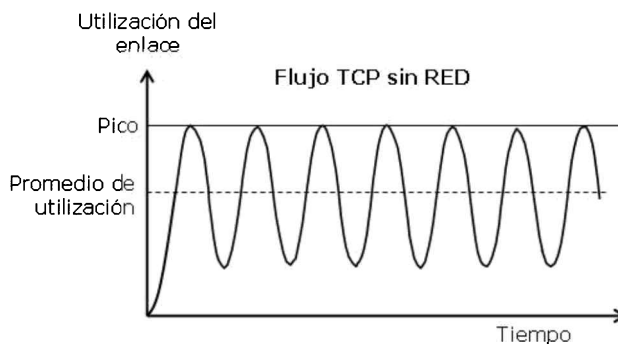
Random Early Detection

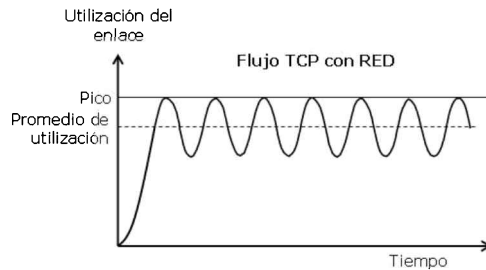
RED (Random Early Detection) es un mecanismo que previene el **tail drop** descartando paquetes de manera aleatoria antes de que éste se produzca. La cantidad de paquetes que son descartados crece a medida que va creciendo el tamaño de la cola de la interfaz. RED no diferencia entre flujos y simplemente descarta aleatoriamente; estadísticamente hablando la idea es que descartará más paquetes provenientes de flujos agresivos.

Ya que RED no descarta paquetes de todos los flujos, no todas las conexiones TCP frenarán y acelerarán al mismo tiempo, provocando la sincronización global. RED sólo es efectivo cuando la mayor parte de los flujos de datos son TCP, ya que el resto de los flujos no disminuirían ante un descarte de paquetes por parte de RED.

RED determina cuándo descartar paquetes basándose en dos valores, el umbral mínimo y el umbral máximo. Cuando el tamaño de la cola es menor que el umbral mínimo no se produce ningún descarte. Si el tamaño de la cola es superior al umbral mínimo e inferior al umbral máximo, se produce un descarte aleatorio de paquetes. Si el tamaño de la cola es superior al umbral máximo, se descartan todos los paquetes. El descarte aleatorio de paquetes que se produce entre el umbral mínimo y máximo es calculado en base a un valor llamado denominador de probabilidad de marca (MPD) y se descartarán 1 de cada cierta cantidad de MDP. Por ejemplo, si el valor del MPD se establece en 10 y el tamaño de la cola tiene establecido un umbral mínimo y máximo, RED descartará 1 de cada 10 paquetes.

Las figuras muestran el comportamiento de un flujo de datos TCP con y sin RED:



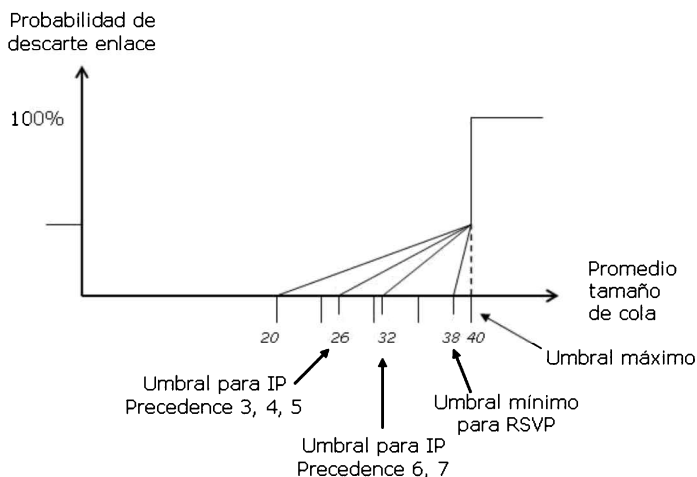


El valor umbral mínimo no debería ser demasiado bajo, para no empezar a descartar paquetes innecesariamente. La diferencia entre el valor mínimo y el valor máximo tampoco debería ser muy baja, para que RED pueda prevenir la sincronización global de TCP.

Weighted Random Early Detection

WRED (Weighted Random Early Detection) funciona de manera similar a RED, pero con la capacidad añadida de poder decidir qué tráfico descartar en caso de que fuera necesario. Con WRED es posible configurar diferentes perfiles (umbral mínimo, máximo y MPD) para dar más prioridad a unos flujos de tráfico que a otros. La prioridad se basa en los valores **IP Precedence** o **DSCP**.

En el siguiente ejemplo el umbral mínimo para tráfico con valores IP Precedence de 3, 4, y 5 está puesto a 26, mientras que el umbral mínimo para tráfico con valores IP Precedence 6 y 7 está puesto a 32. El umbral mínimo para tráfico RSVP es 38. El umbral máximo es 40 para todos y el MPD es 10.



WRED considera el tráfico RSVP sensitivo a los descartes, de manera que el tráfico que no sea RSVP es descartado primero. Por otra parte, los flujos de tráfico no IP son considerados menos importantes que los IP y se empiezan a descartar antes.

WRED no debe aplicarse a colas de tráfico VoIP, ya que dicho tráfico es extremadamente sensitivo a los descartes de paquetes y daría lugar a conversaciones entrecortadas, además de tratarse de tráfico UDP.

Class Based Wighted Random Early Detection

CBWRED (Class Based Wighted Random Early Detection) es el resultado de aplicar **WRED** a las clases de **CBWFQ** descritas en párrafos anteriores.

Configuración de CBWRED

Para habilitar CBWRED en una interfaz se utiliza el comando **random-detect**. Por defecto se emplea IP Precedence, por lo tanto existen 8 perfiles disponibles, mientras que con DSCP existen 64 posibles perfiles. El tráfico que no sea IP será categorizado como valor de IP Precedence 0. CBWRED no puede ser configurado en una interfaz simultáneamente con CQ, PQ o WFQ.

CBWRED se configura a través del comando **random-detect** en cada una de las clases del policy-map. Por cada valor de IP Precedence o DSCP es posible configurar un perfil con un umbral mínimo, máximo y un MPD. Con el tamaño por defecto de la cola de contención (hold-queue) en el rango de 0 a 4096 el umbral mínimo será 1 y el máximo 4096. Por defecto el MPD será 10. Los comandos para configurar WRED basado en DSCP y los perfiles respectivos son los siguientes:

```
Router(router-policy-c)# random-detect dscp-based  
Router(router-policy-c)#random-detect dscp dscp-value min-threshold  
max-threshold mark-prob-denominator  
Router(router-policy-c)#random-detect precedence precedence-value  
min-threshold max-threshold mark-prob-denominator
```

Al aplicar WRED a cada una de las colas se modifica el comportamiento por defecto **tail drop**. Además, dentro de cada cola WRED puede haber un perfil diferente para cada valor de IP Precedence o DSCP, como se muestra en los siguientes ejemplos:

```
class-map Negocios  
match ip precedence 3 4  
class-map Bulk  
match ip precedence 1 2
```

```
!  
policy-map Organizacion  
class Negocios  
bandwidth percent 30  
random-detect  
random-detect precedence 3 26 40 10  
random-detect precedence 4 28 40 10  
class Bulk  
bandwidth percent 20  
random-detect  
random-detect precedence 1 22 36 10  
random-detect precedence 2 24 36 10  
class class-default  
fair-queue  
random-detect  
class-map Negocios  
match ip dscp af21 af22 af23 cs2  
class-map Bulk  
match ip dscp af11 af12 af13 cs1  
!  
policy-map Organizacion  
class Negocios  
bandwidth percent 30  
random-detect dscp-based  
random-detect dscp af21 32 40 10  
random-detect dscp af22 28 40 10  
random-detect dscp af23 24 40 10  
random-detect dscp cs2 22 40 10  
class Bulk  
bandwidth percent 20  
random-detect dscp-based  
random-detect dscp af11 32 36 10  
random-detect dscp af12 28 36 10  
random-detect dscp af13 24 36 10  
random-detect dscp cs1 22 36 10  
class class-default  
fair-queue  
random-detect dscp-based
```

Las estadísticas de los paquetes se pueden ver con el comando **show policy-map interface**:

```
Router# show policy-map interface serial3/1  
Serial3/1  
Service-policy output: sample-policy  
Class-map: voip (match-all)  
0 packets, 0 bytes  
5 minute offered rate 0 bps, drop rate 0 bps  
Match: ip precedence 5  
Weighted Fair Queueing  
Strict Priority  
Output Queue: Conversation 264
```

```

Bandwidth 128 (kbps) Burst 3200 (Bytes)
(pkts matched/bytes matched) 0/0
(total drops/bytes drops) 0/0
Class-map: oro (match-all)
0 packets, 0 bytes
5 minute offered rate 0 bps, drop rate 0 bps
Match: ip precedence 2
Weighted Fair Queueing
Output Queue: Conversation 265
Bandwidth 100 (kbps) Max Threshold 64 (packets)
(pkts matched/bytes matched) 0/0
(depth/total drops/no-buffer drops) 0/0/0
Class-map: plata (match-all)
0 packets, 0 bytes
5 minute offered rate 0 bps, drop rate 0 bps
Match: ip precedence 1
Weighted Fair Queueing
Output Queue: Conversation 266
Bandwidth 80 (kbps)
(pkts matched/bytes matched) 0/0
(depth/total drops/no-buffer drops) 0/0/0
exponential weight: 9
mean queue depth: 0
class Transmitted Random drop Tail drop Minimum Maximum Mark
pkts/bytes pkts/bytes pkts/bytes thresh thresh prob
0      0/0    0/0    0/0    20     40     1/10
1      0/0    0/0    0/0    22     40     1/10
2      0/0    0/0    0/0    24     40     1/10
3      0/0    0/0    0/0    26     40     1/10
4      0/0    0/0    0/0    28     40     1/10
5      0/0    0/0    0/0    30     40     1/10
6      0/0    0/0    0/0    32     40     1/10
7      0/0    0/0    0/0    34     40     1/10
rsvp   0/0    0/0    0/0    36     40     1/10
Class-map: class-default (match-any)
0 packets, 0 bytes
5 minute offered rate 0 bps, drop rate 0 bps
Match: any

```